

A MAJOR PROJECT STAGE-II REPORT
ON
**FRAUD DETECTION AND ANALYSIS OF INSURANCE CLAIM
USING MACHINE LEARNING**

Submitted in the partial fulfillment of the requirements for the award of

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

**SUBMITTED BY
GULLA HARITHA
22BK1A0568**

**Under the guidance of
Mr. P Satish M.Tech
Assistant Professor**



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
St. Peter's Engineering College (UGC Autonomous)
Approved by AICTE, New Delhi, and NAAC Accredited with 'A' Grade,
Affiliated to JNTU, Hyderabad, Telangana.
2025-2026**



**St. PETER'S
ENGINEERING COLLEGE**
(UGC - AUTONOMOUS | Affiliated to JNTU Hyderabad)
Approved by AICTE, An ISO Certified Institution



nirf INNOVATION
India Ranking 2023
151-300 Rank Band

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that a Project entitled “**FRAUD DETECTION AND ANALYSIS OF INSURANCE CLAIM USING MACHINE LEARNING**” is carried out by **GULLA HARITHA (22BK1A0568)** in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of bonafide work done by team under my supervision during the academic year 2025– 2026.

INTERNAL GUIDE

Mr. P Satish
Assistant Professor
Department of CSE

HEAD OF THE DEPARTMENT

Dr.E.Madhusudhana Reddy
Professor &HOD
Department of CSE

PROJECT COORDINATOR

Mr. A. Senthil Murugan M.E,M.B.A,(Ph.D)
Assistant Professor
Department of CSE

EXTERNAL EXAMINER



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING ACKNOWLEDGEMENT

I sincerely express our deep sense of gratitude to **Mr. P Satish**, for her valuable guidance, encouragement and cooperation during all phases of the project.

I am greatly indebted to our Project Coordinator **Mr. A. Senthil Murugan**, for providing valuable advice, constructive suggestions and encouragement without whom it would not been possible to complete this project.

It is a great opportunity to render our sincere thanks to **Dr.E.Madhusudhana Reddy**, Head of the Department, Computer Science and Engineering for his timely guidance and highly interactive attitude which helped us a lot in successful execution of the Project.

We are extremely thankful to our Principal **Dr. Koduganti Venkata Rao**, who stood as an inspiration behind this project and heartfelt for his endorsement and valuable suggestions.

We respect and thank our Administrative Director **Mr. T. Anuraag Reddy**, Academic Director **Mrs. T. Saroja Reddy** and secretary **Sri. T. V. Reddy**, for providing us an opportunity to do the project work at **St. PETER'S ENGINEERING COLLEGE** and we are extremely thankful to them for providing such a nice support and guidance which made us to complete the project.

We also acknowledge with a deep sense of reverence, our gratitude towards our parents, who have always supported us morally as well as economically. We also express gratitude to all our friends' who have directly or indirectly helped us to complete this project work. We hope that we can build up on the experience and knowledge that we have gained and make a valuable contribution towards the growth of the society in coming future.

GULLA HARITHA (22BK1A0568)



**St. PETER'S
ENGINEERING COLLEGE**
(UGC - AUTONOMOUS | Affiliated to JNTU Hyderabad)
Approved by AICTE, An ISO Certified Institution



INNOVATION
India Ranking 2023
151-300 Rank Band

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INSTITUTE VISION

To be a renowned Educational Institution that moulds Students into Skilled Professionals fostering Technological Development, Research and Entrepreneurship meeting the societal needs.

INSTITUTE MISSION

IM1: Making students knowledgeable in the field of core and applied areas of Engineering to innovate Technological solutions to the problems in the Society.

IM2: Training the Students to impart the skills in cutting edge technologies, with the help of relevant stake holders.

IM3: Fostering a conducive ambience that inculcates research attitude, identifying promising fields for entrepreneurship with ethical, moral and social responsibilities.



**St. PETER'S
ENGINEERING COLLEGE**
(UGC - AUTONOMOUS | Affiliated to JNTU Hyderabad)
Approved by AICTE, An ISO Certified Institution



nirf INNOVATION
India Ranking 2023
151-300 Rank Band

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT VISION

To be a global leader in Computer Science Education, Research and Development, Innovation through industry collaboration and cultivating future-ready technologists.

DEPARTMENT MISSION

DM1: Cultivate experiential learning through state-of-the-art technologies and algorithms to address real-world interdisciplinary challenges.

DM2: Collaborate with industry and academia to equip students with cutting-edge skills and prepare them for dynamic careers.

DM3: Strengthen theoretical and applied research in Computer Science & Engineering, fostering innovation, impactful contributions, and instilling ethical values and social responsibility in all endeavors.



**St. PETER'S
ENGINEERING COLLEGE**
(UGC - AUTONOMOUS | Affiliated to JNTU Hyderabad)
Approved by AICTE, An ISO Certified Institution



nirf INNOVATION
India Ranking 2023
151-300 Rank Band

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

PEO1: Innovate and contribute to research, driving technological advancements in industry and society.

PEO2: Excel as leaders in software development, ensuring industry and government excellence.

PEO3: Lead entrepreneurial ventures, fostering growth in computer science-driven technologies at national and global levels.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING **PROGRAM OUTCOMES (POs)**

Engineering Graduates will be able to:

PO1. Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. Individual and teamwork: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. Communication: Communicate effectively on complex engineering activities with the

engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.⁶

PO11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



**St. PETER'S
ENGINEERING COLLEGE**
(UGC - AUTONOMOUS | Affiliated to JNTU Hyderabad)
Approved by AICTE, An ISO Certified Institution



nirf INNOVATION
India Ranking 2023
151-300 Rank Band

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAM SPECIFIC OBJECTIVES (PSO'S)

PSO-1: Design and develop computing subsystems for data storage, communication, information processing, and knowledge discovery.

PSO-2: Design algorithms for real-world problems, focusing on execution and complexity analysis while considering security, cost, quality, and privacy parameters in software development.



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I declare that a Major Project entitled **“FRAUD DETECTION AND ANALYSIS OF INSURANCE CLAIM USING MACHINE LEARNING”** is an Original Work submitted by the following group members who have actively contributed and submitted in partial fulfillment for the award of degree in **“Bachelor of Technology in Computer Science and Engineering”**, at **St. Peter's Engineering College, Hyderabad**, and this project work has not been submitted by me to any other college or university for the award of any kind of degree.

Group No: 03

Program: B.Tech

Branch & Section: CSE

Major Project Title: FRAUD DETECTION AND ANALYSIS OF INSURANCE CLAIM USING MACHINE LEARNING

Date Submitted:

Name	Roll Number	Signature
GULLA HARITHA	22BK1A0568	

TABLE OF CONTENT		
S.NO	Topic	Page No
1.	Abstract	1
2.	Introduction	2-3
3.	Problem Statement	4-6
4.	Proposed System	7-10
5.	Key Features	11-15
6.	Existing Solutions	16-19
7.	Literature Survey	20-23
8.	Existing Solutions Vs Proposed System	24-26
9.	Methodology	27-30
10.	System Requirements	31-34
11.	Implementation	35-53
12.	UML Diagrams	54-60
13.	Output's	61-68
14.	Benefits And Limitations	69-71
15.	Conclusion	72-73
16.	Reference	74-75

ABSTRACT

Insurance companies, as major commercial enterprises, have been increasingly facing challenges due to the rapid growth of fraudulent claims in recent years. Fraudulent activities in insurance not only result in significant financial losses but also affect the overall efficiency and trustworthiness of the system. These frauds occur across various types of insurance such as health, life, and automobile insurance. Among these, automobile insurance fraud is one of the most common and severe types, where individuals submit false or exaggerated claims, often involving fake accident reports or manipulated damage assessments. The increasing number of such fraudulent claims creates a need for efficient and automated fraud detection systems.

This project aims to develop a machine learning-based system for detecting and analyzing fraudulent insurance claims. The system uses a structured insurance claim dataset containing multiple attributes such as claim amount, customer details, accident information, and policy data. By analyzing these features, the model identifies patterns and relationships that distinguish fraudulent claims from genuine ones. Data preprocessing techniques such as handling missing values, normalization, and feature selection are applied to improve data quality and model performance.

Multiple machine learning classification algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine are implemented to build predictive models. These models are trained and tested on the dataset to classify claims as either fraudulent or legitimate. A comparative analysis is performed to evaluate the effectiveness of each algorithm using performance metrics such as accuracy, precision, recall, and F1-score. The confusion matrix is also used to visualize the performance of each model in terms of correct and incorrect classifications.

To efficiently handle large datasets and improve processing speed, the project utilizes the PySpark framework in Python. PySpark enables distributed data processing, making the system scalable and suitable for real-world applications involving big data. The integration of machine learning with big data technologies enhances the reliability and efficiency of fraud detection.

CHAPTER 1

INTRODUCTION

INTRODUCTION

The insurance industry plays a vital role in providing financial protection against unforeseen risks and losses. However, with the rapid growth of the insurance sector, there has also been a significant increase in fraudulent activities, particularly in the area of insurance claims. Fraudulent claims involve intentional deception by policyholders to gain illegitimate financial benefits. These activities not only cause substantial financial losses to insurance companies but also lead to higher premiums for genuine customers and reduce overall trust in the system.

Insurance fraud can occur in various forms, including false claims, exaggerated damages, duplicate claims, and staged incidents. Among these, automobile insurance fraud is one of the most prevalent types, where individuals submit fake accident reports or manipulate claim details to receive higher compensation. Detecting such fraud manually is a complex and time-consuming process due to the large volume of data and the subtle nature of fraudulent patterns.

With the advancement of technology, machine learning has emerged as an effective solution for fraud detection. Machine learning algorithms can analyze large datasets, identify hidden patterns, and make accurate predictions based on historical data. By training models on past insurance claim records, it becomes possible to classify new claims as either genuine or fraudulent with improved accuracy.

This project focuses on developing a machine learning-based system for the detection and analysis of fraudulent insurance claims. The system uses various classification algorithms to build predictive models and evaluate their performance using standard metrics such as accuracy, precision, recall, and confusion matrix. Additionally, the use of PySpark enables efficient processing of large-scale datasets, making the system scalable and suitable for real-world applications.

The main objective of this project is to provide an intelligent, automated, and reliable solution for fraud detection that helps insurance companies minimize losses, improve operational efficiency, and enhance decision-making processes.

CHAPTER 2

PROBLEM STATEMENT

PROBLEM STATEMENT

The insurance industry has experienced rapid growth over the past few decades, offering financial protection and risk management solutions to individuals and organizations. However, along with this expansion, the industry has also witnessed a significant rise in fraudulent activities, particularly in the area of insurance claims. Fraudulent claims have become a major concern for insurance companies, as they lead to substantial financial losses, increased operational costs, and reduced trust among genuine policyholders.

Insurance fraud occurs when a policyholder deliberately deceives the insurance company to obtain illegitimate financial benefits. This deception can take various forms, such as submitting false claims, exaggerating damages, staging accidents, or providing misleading information during the claim process. Among all types of insurance fraud, automobile insurance fraud is one of the most prevalent and severe, where individuals often create fake accident scenarios or manipulate claim details to receive higher compensation. These fraudulent practices are becoming more sophisticated, making them difficult to detect using traditional methods.

Currently, most insurance companies rely on manual verification processes and rule-based systems to detect fraudulent claims. These methods involve human intervention, predefined rules, and basic statistical analysis. While these approaches may identify some obvious fraud cases, they are often inefficient, time-consuming, and prone to errors. As the volume of insurance claims increases, it becomes increasingly difficult for human experts to analyze each claim thoroughly. Moreover, fraudsters continuously adapt their strategies to bypass existing detection mechanisms, making rule-based systems less effective over time.

Another major challenge in fraud detection is the complexity and high dimensionality of insurance data. Insurance claim datasets typically contain a large number of features, including customer details, policy information, claim history, accident reports, and financial data. Identifying meaningful patterns and relationships within such complex datasets is not feasible using traditional analytical methods. Additionally, the presence of noisy, incomplete, or imbalanced data further complicates the detection process. In many cases, fraudulent claims represent only a small percentage of the total data, making it difficult to build accurate

detection models.

To address these challenges, there is a need for an advanced, automated, and data-driven approach to fraud detection. Machine learning techniques provide a powerful solution by enabling systems to learn from historical data and identify hidden patterns associated with fraudulent behavior. These algorithms can process large volumes of data efficiently, adapt to new patterns, and improve their performance over time. However, selecting the most suitable machine learning model and evaluating its effectiveness remains a critical task.

The problem addressed in this project is the development of an efficient and reliable system for detecting fraudulent insurance claims using machine learning techniques. The system must be capable of analyzing large-scale insurance datasets, identifying suspicious patterns, and accurately classifying claims as either genuine or fraudulent. It should also handle issues such as data imbalance, missing values, and feature selection to ensure optimal performance. Furthermore, there is a need to perform a comparative analysis of different machine learning classification algorithms to determine which model provides the best results for fraud detection. This involves evaluating each model using performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix. Such analysis helps in understanding the strengths and limitations of each algorithm and selecting the most appropriate model for real-world implementation.

In addition to model development, the system must be scalable and capable of handling large datasets efficiently. Traditional data processing techniques may not be sufficient for dealing with big data in real-time applications. Therefore, the use of distributed computing frameworks such as PySpark becomes essential. PySpark enables parallel processing of large datasets, reducing computation time and improving overall system performance.

CHAPTER 3

PROPOSED SYSTEM

PROPOSED SYSTEM

The proposed system aims to develop an efficient and intelligent fraud detection model for insurance claims using machine learning techniques. The primary objective of the system is to improve predictive performance and achieve higher accuracy in detecting fraudulent claims by applying advanced feature engineering, feature selection, and parameter tuning techniques. In this system, special focus is given to handling imbalanced datasets, as fraudulent claims typically represent a small portion of the total data. Various machine learning algorithms are explored and evaluated to enhance the detection accuracy and reliability of the system. The influence of feature engineering and parameter optimization is studied to identify the best-performing model.

The dataset used in this project is divided into three main segments: training set, validation set, and testing set. Initially, the model is trained using a portion of the dataset (training set). The model parameters are then fine-tuned using the validation set to improve performance. Finally, the optimized model is evaluated on the testing dataset to measure its real-world effectiveness. To ensure consistency and robustness, the models are tested using multiple random splits of the dataset.

The overall system architecture consists of the following three major layers:

1. Data Pre-processing Layer

In this layer, the raw insurance claim data is prepared for analysis and model building. The preprocessing steps include:

- Cleaning the dataset by handling missing or inconsistent values
- Extracting dependent (target) and independent (feature) variables
- Encoding categorical variables into numerical form
- Splitting the dataset into training and testing sets using the train-test split method from the *sklearn* library
- Applying feature scaling techniques such as normalization or standardization improve

This step ensures that the data is structured and suitable for machine learning algorithms.

2. Model Training and Prediction Layer

In this layer, machine learning algorithms are applied to build predictive models. Logistic Regression is primarily used as a classification algorithm for fraud detection.

- The Logistic Regression model is implemented using the *LogisticRegression* class from the *sklearn* library
- A classifier object is created and trained using the training dataset
- The trained model learns patterns and relationships between input features and the target variable (fraud or non-fraud)
- Once the model is trained, it is used to predict outcomes on the test dataset

This layer is responsible for identifying whether an insurance claim is fraudulent or genuine.

3. Evaluation and Visualization Layer

This layer focuses on evaluating the performance of the trained model and improving it further.

- The model performance is evaluated using a confusion matrix
- Metrics such as accuracy, precision, recall, and F1-score are calculated
- The confusion matrix helps determine whether fraudulent claims are correctly classified
- Model parameters are adjusted and experiments are repeated to improve performance
- Different machine learning algorithms and feature sets are tested for better results
- Visualization techniques are used to analyze and compare the model performance

This iterative process helps in achieving optimal model performance and reliability.

Advantages of the Proposed System

- Multiple machine learning models are tested and compared on the dataset
- Feature engineering techniques are applied to improve model performance
- The system supports iterative improvement through parameter tuning and repeated

testing

- It effectively handles binary classification (fraudulent vs. non-fraudulent claims)
- The model provides accurate and reliable predictions for fraud detection
- The system is scalable and can be extended to large datasets using advanced tools

CHAPTER 4

KEY FEATURES

KEY FEATURES

The proposed system for fraud detection and analysis of insurance claims using machine learning is designed with several advanced features to ensure high accuracy, efficiency, and scalability. These features focus on improving fraud detection capability, handling large datasets, and providing reliable predictions. The key features of the system are explained in detail below:

1. Automated Fraud Detection System

One of the primary features of the system is automation. Traditional fraud detection methods rely heavily on manual verification, which is time-consuming and prone to human error. The proposed system automates the entire fraud detection process using machine learning models. Once trained, the system can instantly analyze new insurance claims and classify them as fraudulent or genuine without human intervention. This significantly reduces processing time and operational costs while increasing efficiency.

2. Use of Multiple Machine Learning Algorithms

The system incorporates various machine learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and Support Vector Machine for classification tasks. Each algorithm has its own strengths and is suitable for different types of data patterns. By implementing multiple algorithms, the system allows comparative analysis to determine which model performs best for fraud detection. This improves the robustness and reliability of the final model.

3. Handling Imbalanced Datasets

Insurance fraud detection is a classic example of an imbalanced dataset problem, where fraudulent cases are much fewer compared to genuine claims. This imbalance can lead to biased models that favor the majority class. The proposed system addresses this issue by

applying techniques such as resampling (oversampling/undersampling) and adjusting class weights. These methods ensure that the model learns patterns from both classes effectively and improves its ability to detect rare fraudulent cases.

4. Data Preprocessing and Cleaning

Raw insurance data often contains missing values, noise, and inconsistencies. Data preprocessing is an essential step to prepare the dataset for machine learning. The system performs various preprocessing operations such as:

- Handling missing or null values
- Removing duplicate or irrelevant data
- Converting categorical data into numerical form using encoding techniques
- Normalizing or standardizing numerical features

These steps improve the quality of the data and ensure that the machine learning models produce accurate results.

5. Feature Engineering

Feature engineering plays a crucial role in improving model performance. In this system, new features are created from existing data to better represent the underlying patterns. For example, combining multiple attributes or extracting meaningful information from raw data can enhance the predictive power of the model. Proper feature engineering helps the model understand complex relationships within the dataset and improves fraud detection accuracy.

6. Feature Selection Optimization

Not all features in a dataset contribute equally to the prediction process. Some features may be irrelevant or redundant, which can negatively affect model performance. The proposed system uses feature selection techniques to identify the most important features that influence fraud detection. This reduces the dimensionality of the dataset, speeds up computation, and improves model accuracy.

7. Training, Validation, and Testing Approach

To ensure reliable and unbiased results, the dataset is divided into three parts:

- **Training Set:** Used to train the machine learning model
- **Validation Set:** Used to tune model parameters and avoid overfitting
- **Testing Set:** Used to evaluate the final performance of the model

This structured approach ensures that the model performs well on unseen data and provides realistic performance evaluation.

8. Model Evaluation Using Performance Metrics

The system evaluates the performance of different machine learning models using standard metrics such as:

- **Accuracy:** Measures overall correctness of predictions
- **Precision:** Indicates how many predicted fraud cases are actually fraud
- **Recall:** Measures how many actual fraud cases are correctly identified
- **F1-Score:** Balances precision and recall
- **Confusion Matrix:** Provides a detailed breakdown of predictions

These metrics help in understanding the strengths and weaknesses of each model and selecting the best-performing one.

9. Confusion Matrix Analysis

The confusion matrix is a key tool used in this system to evaluate classification performance.

It provides a clear visualization of:

- True Positives (correct fraud detection)
- True Negatives (correct non-fraud detection)
- False Positives (incorrect fraud detection)
- False Negatives (missed fraud cases)

This analysis helps in identifying errors and improving the model to reduce false predictions.

10. Parameter Tuning and Iterative Improvement

The system follows an iterative approach to improve model performance. Model parameters are adjusted and tested multiple times using techniques such as hyperparameter tuning. By experimenting with different parameter values and configurations, the system identifies the optimal settings that maximize accuracy and minimize errors.

11. Scalable Data Processing with PySpark

To handle large volumes of insurance data efficiently, the system uses PySpark, a powerful big data processing framework. PySpark enables distributed computing, allowing data to be processed in parallel across multiple nodes. This improves processing speed and makes the system scalable for real-world applications involving large datasets.

12. Binary Classification Output

The system performs a binary classification task, where each insurance claim is classified into one of two categories:

- **Fraudulent (Yes)**
- **Non-Fraudulent (No)**

This simple and clear output makes it easy for insurance companies to take quick and effective decisions regarding claim approval or investigation.

13. Visualization of Results

Visualization plays an important role in understanding model performance. The system uses graphical representations such as charts and confusion matrices to display results. These visualizations help users analyze trends, compare models, and interpret results more effectively.

14. Consistency and Reliability of Results

To ensure that the model produces consistent results, the system tests the model using multiple random splits of the dataset.

CHAPTER 5

EXISTING SOLUTIONS

EXISTING SOLUTIONS

Fraud detection in insurance claims has been a critical area of concern for many years. Before the adoption of advanced machine learning techniques, insurance companies relied on traditional methods and basic analytical systems to identify fraudulent activities. Although these methods provided some level of fraud detection, they had several limitations in terms of accuracy, scalability, and efficiency. The existing solutions for fraud detection in insurance claims can be broadly categorized as follows:

1. Manual Verification System

One of the earliest and most commonly used approaches in fraud detection is manual verification. In this method, insurance agents and investigators review each claim individually to determine its authenticity.

- Claims are analyzed based on documents, customer history, and supporting evidence
- Experts use their experience and judgment to identify suspicious patterns
- Investigations may include field verification and interviews

Limitations:

- Time-consuming and labor-intensive
- Prone to human errors and bias
- Not scalable for large volumes of claims
- Difficult to detect complex fraud patterns

2. Rule-Based Systems

Rule-based systems are widely used in traditional fraud detection. These systems operate based on predefined rules and conditions set by domain experts.

- Rules are created based on known fraud patterns (e.g., high claim amount, repeated claims, suspicious timing)
- If a claim satisfies certain conditions, it is flagged as suspicious
- Easy to implement and understand

Limitations:

- Cannot detect new or evolving fraud patterns
- Requires frequent updates and maintenance
- High false positives (flagging genuine claims as fraud)
- Limited flexibility and adaptability

3. Statistical and Data Analysis Methods

Statistical techniques were introduced to improve fraud detection by analyzing historical data.

- Use of descriptive statistics and probability models
- Identification of outliers and anomalies in claim data
- Pattern recognition using basic statistical tools

Limitations:

- Limited ability to handle complex and high-dimensional data
- Less effective for large datasets
- Cannot capture non-linear relationships
- Requires strong assumptions about data distribution

4. Database Query-Based Systems

Some systems rely on database queries to identify suspicious claims.

- Queries are written to filter claims based on specific criteria
- Example: detecting duplicate claims or unusually high claim amounts
- Used in combination with rule-based systems

Limitations:

- Limited intelligence and automation
- Dependent on predefined queries
- Not suitable for dynamic fraud detection

5. Traditional Machine Learning Approaches

Before advanced models, basic machine learning techniques such as Logistic Regression and

Naïve Bayes were used.

- Models trained on historical claim data
- Used for binary classification (fraud or non-fraud)
- Provided better results compared to rule-based systems

Limitations:

- Limited performance on highly imbalanced datasets
- Requires extensive feature engineering
- May not perform well on large-scale data without optimization

6. Data Mining Techniques

Data mining methods have been applied to extract patterns and knowledge from insurance datasets.

- Techniques include clustering, classification, and association rule mining
- Helps in identifying hidden patterns and relationships
- Used for exploratory analysis

Limitations:

- Requires domain expertise for interpretation
- Computationally expensive for large datasets
- Limited real-time processing capability

7. Basic Fraud Detection Software Tools

Some insurance companies use commercial fraud detection tools and software.

- Provide predefined modules for fraud analysis
- Include reporting and visualization features
- Can integrate with existing systems

Limitations:

- Expensive to implement and maintain
- Limited customization options
- May not be suitable for specific datasets or requirements

CHAPTER 6

LITERATURE SURVEY

LITERATURE SURVEY

Insurance fraud detection has been extensively studied in recent years, with a growing emphasis on machine learning techniques due to the high volume and complexity of claims data. The literature can be categorized into traditional machine learning approaches, deep learning approaches, and hybrid/optimization-based methods.

A. Traditional Machine Learning Approaches

Traditional machine learning techniques such as Logistic Regression, Naive Bayes, Decision Trees, and Random Forests have been widely used for fraud detection due to their simplicity and interpretability. Almeida et al. studied SMS spam and insurance fraud using Naive Bayes and Logistic Regression, demonstrating that probabilistic and linear models can detect fraudulent patterns effectively in structured datasets. Similarly, Chen and Li applied Decision Trees and Random Forest classifiers to insurance datasets, reporting high accuracy when features were carefully engineered and balanced.

Limitations: These approaches often struggle with highly unbalanced datasets, where fraudulent claims constitute a small fraction of the total claims. Additionally, manual feature engineering and parameter tuning are required for optimal performance.

B. Deep Learning Approaches

Recent studies have explored deep learning models for fraud detection, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and LSTMs. These models can automatically learn complex patterns from large datasets and handle sequential dependencies in claim histories. For instance, Jurgovsky et al. used RNNs to detect fraud in financial transactions and demonstrated improved recall on unbalanced datasets.

Limitations: Despite their performance, deep learning models require large labeled datasets and high computational resources. They are often less interpretable, which is

critical in regulatory environments like insurance.

C. Hybrid and Optimization-Based Approaches

Hybrid methods combine feature engineering, ensemble classifiers, and optimization algorithms to improve detection rates. For example, some studies integrate SMOTE (Synthetic Minority Oversampling Technique) with ensemble classifiers to address class imbalance. Others optimize model hyperparameters using genetic algorithms or grid search, improving detection accuracy across multiple random splits .

Advantages: Hybrid approaches are robust against dataset imbalance and can capture subtle fraudulent patterns. They also allow comparative evaluation of multiple models, which is valuable in identifying the best-performing algorithm for a specific dataset.

D. PySpark and Scalable Machine Learning

With increasing data volumes, PySpark has emerged as a scalable framework for machine learning on large insurance datasets. It enables distributed data processing, feature engineering, and model training across clusters. Studies by Zhang et al. [6] demonstrated that PySpark-based pipelines reduce training time significantly while maintaining high accuracy, making it suitable for enterprise-level insurance fraud detection.

Research Gap:

1. Most prior work focuses on either traditional ML or deep learning but rarely compares multiple ML models on insurance datasets using scalable frameworks.
2. Handling class imbalance and evasive claim patterns remains a challenge.
3. Few studies provide a complete end-to-end pipeline, including preprocessing, model evaluation, and visualization, suitable for real-world deployment.

feature importance ranking are commonly used to identify the most influential variables in fraud detection.

A major challenge discussed in many studies is the issue of imbalanced datasets. In insurance

fraud detection, fraudulent claims are very few compared to genuine ones. This imbalance can lead to biased models that fail to detect fraud effectively. To overcome this problem, researchers have proposed techniques such as oversampling, undersampling, and synthetic data generation methods like SMOTE. These techniques help in balancing the dataset and improving the detection of fraudulent cases.

In recent years, deep learning techniques have also been explored for fraud detection. Models such as neural networks, convolutional neural networks (CNN), and recurrent neural networks (RNN) are capable of capturing complex patterns in large datasets. Although these models provide high accuracy, they require large amounts of data and computational resources. Additionally, their lack of interpretability makes them difficult to use in critical applications like insurance, where understanding the decision-making process is important.

The use of big data technologies has further enhanced fraud detection systems. Frameworks like PySpark enable efficient processing of large datasets by distributing the computation across multiple systems. This makes it possible to handle real-world insurance data, which is often large and complex. Combining machine learning with big data tools improves both speed and scalability of fraud detection systems.

CHAPTER 7

EXISTING VS PRAPOSED SYSTEM

EXISTING VS PROPOSED SYSTEM

The existing system for fraud detection in insurance claims mainly relies on manual verification processes and rule-based approaches. In this method, insurance officers and investigators examine each claim individually using predefined rules and their domain knowledge. These rules are usually based on past fraud cases, such as unusually high claim amounts, repeated claims, or suspicious customer behavior. While this approach can identify some obvious fraudulent activities, it has several limitations. It is time-consuming, requires significant human effort, and is prone to errors and bias. Moreover, as the volume of insurance claims increases, it becomes difficult to manually analyze each case efficiently. Another major drawback is that rule-based systems cannot adapt to new and evolving fraud patterns, making them less effective over time.

In contrast, the proposed system introduces a machine learning-based approach to automate and improve the fraud detection process. Instead of relying on fixed rules, the system learns patterns from historical data and uses this knowledge to predict whether a new claim is fraudulent or genuine. Machine learning algorithms such as Logistic Regression, Decision Tree, and Random Forest are used to build predictive models. These models are capable of identifying complex and hidden patterns that are not easily detectable through manual methods. As a result, the proposed system provides higher accuracy and more reliable predictions.

Another key difference lies in data handling and processing. The existing system has limited capability to process large volumes of data, whereas the proposed system uses advanced data preprocessing techniques such as data cleaning, feature engineering, and feature scaling to improve data quality. Additionally, by using technologies like PySpark, the proposed system can efficiently handle large datasets and perform distributed processing, making it suitable for real-world applications.

The proposed system also improves adaptability and scalability. While the existing system

struggles to handle new types of fraud, the machine learning model can continuously learn and adapt as new data becomes available. This makes the system more flexible and capable of detecting emerging fraud patterns. Furthermore, the automated nature of the proposed system significantly reduces processing time and minimizes the need for human intervention. In terms of performance evaluation, the existing system lacks proper metrics to measure effectiveness, whereas the proposed system uses standard evaluation techniques such as accuracy, precision, recall, F1-score, and confusion matrix. These metrics provide a clear understanding of model performance and help in improving the system further.

Overall, the proposed system offers a more efficient, accurate, and scalable solution compared to the existing system. It overcomes the limitations of traditional methods by providing an intelligent and data-driven approach to fraud detection. This not only reduces financial losses for insurance companies but also enhances decision-making and operational efficiency.

Key Improvements

The proposed system offers several improvements over the existing system. It reduces the need for manual intervention by automating the fraud detection process. It also provides higher accuracy by using advanced machine learning algorithms. Unlike rule-based systems, it can adapt to new fraud patterns by learning from updated data.

Another important advantage is the use of performance evaluation metrics such as accuracy, precision, recall, and confusion matrix. These metrics help in measuring and improving the effectiveness of the model. Additionally, the system ensures consistency in results by testing the model on different data splits.

CHAPTER 8

METHODOLOGY

METHODOLOGY

The methodology of the proposed system describes the step-by-step process used to develop an efficient fraud detection model for insurance claims. It involves data collection, preprocessing, model building, evaluation, and result analysis. The system follows a structured approach to ensure accuracy, reliability, and scalability.

Data Collection

The first step in the methodology is collecting the insurance claim dataset. The dataset contains various attributes such as customer details, policy information, claim amount, accident details, and claim history. This data serves as the foundation for training and testing the machine learning models. A good quality dataset is essential for building an effective fraud detection system.

Data Preprocessing

Raw data is often incomplete, inconsistent, and noisy. Therefore, preprocessing is performed to clean and prepare the data for analysis. This step includes handling missing values, removing duplicate records, and correcting inconsistencies. Categorical variables are converted into numerical form using encoding techniques.

The dataset is then divided into dependent (target variable) and independent variables (features). Feature scaling methods such as normalization or standardization are applied to ensure that all features contribute equally to the model. Finally, the dataset is split into training and testing sets using the train-test split method.

Feature Engineering and Selection

In this stage, new features are created from existing data to improve model performance.

Feature engineering helps in extracting meaningful information that can better represent fraud patterns. Additionally, feature selection techniques are applied to identify the most relevant features and remove unnecessary ones.

Model Building

The processed data is used to train machine learning models. Algorithms such as Logistic Regression, Decision Tree, and Random Forest are applied to classify insurance claims as fraudulent or non-fraudulent. The models learn patterns from the training data and build a relationship between input features and the target variable.

Among these, Logistic Regression is commonly used as a baseline model due to its simplicity and effectiveness in binary classification problems.

Model Validation and Testing

After training, the model is validated using a validation dataset to fine-tune parameters and avoid overfitting. Once the model is optimized, it is tested on unseen data (testing set) to evaluate its real-world performance. Multiple random splits of data may be used to ensure consistency and reliability of results.

Performance Evaluation

The performance of the model is evaluated using various metrics such as accuracy, precision, recall, and F1-score. A confusion matrix is used to analyze the classification results in detail, showing correct and incorrect predictions.

This step helps in identifying how well the model detects fraudulent claims and whether any improvements are needed.

Visualization and Analysis

The results are visualized using graphs and charts to better understand model performance. Visualization helps in comparing different models and identifying trends in the data. Based on the analysis, model parameters may be adjusted and the process is repeated to achieve better results.

Implementation Using PySpark

To handle large datasets efficiently, the system uses PySpark for distributed data processing. PySpark enables faster computation and scalability, making the system suitable for real-world insurance applications where data is large and continuously growing.

The methodology adopted in this project follows a systematic and iterative approach to ensure accurate and reliable fraud detection. The process begins with understanding the nature of the insurance dataset and identifying the key attributes that influence fraudulent behavior. Since insurance data is often complex and contains both numerical and categorical features, careful handling of data becomes essential for building an effective model.

A significant part of the methodology involves data preparation, where raw data is transformed into a suitable format for analysis. During this phase, special attention is given to handling missing values and inconsistencies, as poor data quality can directly affect model performance. Encoding techniques such as label encoding or one-hot encoding are applied to convert categorical variables into numerical form, enabling machine learning algorithms to process them efficiently. Feature scaling is also performed to bring all variables to a common scale, preventing bias toward features with larger values.

Another important aspect of the methodology is dealing with class imbalance. Since fraudulent claims are usually much fewer than genuine claims, the dataset tends to be skewed. This imbalance can lead to poor model performance if not addressed properly. Techniques such as oversampling the minority class or undersampling the majority class are applied to create a balanced dataset. This ensures that the model learns patterns from both fraudulent and non-fraudulent cases effectively.

CHAPTER 9

SYSTEM REQUIREMENTS

SYSTEM REQUIREMENTS

The Fraud Detection and Analysis of Insurance Claims system requires a standard computing environment to ensure smooth development and execution. The system should have a processor such as Intel Pentium-IV or higher with a minimum of 4 GB RAM and at least 20 GB of free disk space to handle data processing and machine learning operations efficiently.

The system should run on an operating system like Windows 7, Windows 10, or Windows 11. The application is developed using Python as the programming language, along with web technologies such as HTML, CSS, and JavaScript for the user interface. The backend is supported using the Django framework, and MySQL is used as the database for storing user and insurance data.

Hardware Requirements

The following hardware configuration is required for the implementation of the system:

- **Processor** : Pentium-IV or higher
- **RAM** : Minimum 4 GB
- **Hard Disk** : Minimum 20 GB of free space
- **Keyboard** : Standard Windows Keyboard
- **Mouse** : Two or Three Button Mouse
- **Monitor** : SVGA Monitor
- The hardware configuration plays an important role in determining the performance of the system. Although the proposed system does not require very high-end infrastructure, a standard computing environment is sufficient for development and execution.
- The processor is one of the most important components. A **Pentium-IV processor or higher** is required to execute the system efficiently. However, modern processors such as Intel i3, i5, or above are recommended for faster computation, especially when dealing with large datasets and machine learning operations.

- The system requires a minimum of **4 GB RAM** to ensure smooth functioning of the application. RAM is essential for running multiple processes such as data preprocessing, model training, and visualization simultaneously. For better performance and faster execution, higher RAM (8 GB or more) can be used.
- A minimum of **20 GB hard disk space** is required to store datasets, program files, libraries, and output results. As the dataset size increases, additional storage may be required. Solid State Drives (SSD) are preferred over traditional hard drives for faster data access and improved system performance.
- Input and output devices are also necessary for user interaction. A **standard Windows keyboard** is used for entering commands and interacting with the system. A **two or three button mouse** helps in navigation and selection of options. A **SVGA monitor** .

Software Requirements

The software requirements for developing and running the system are as follows:

- **Operating System** : Windows 7 Ultimate or above
- **Programming Language** : Python
- **Front-End** : Python (for interface handling)
- **Back-End** : Django ORM
- **Designing Tools** : HTML, CSS, JavaScript
- **Database** : MySQL (WAMP Server)
- Software requirements are crucial for the development and execution of the fraud detection system. The project uses a combination of programming languages, frameworks, and tools to implement machine learning models and manage data efficiently.
- The system requires an **operating system such as Windows 7 Ultimate or above**. Modern versions like Windows 10 or Windows 11 are also compatible and provide better performance and security.
- The primary programming language used in this project is **Python**, which is widely used for machine learning and data analysis. Python provides a simple syntax and a large number of libraries that make it suitable for developing predictive models.

- For the front-end, Python is used to handle the application interface and logic. Additionally, **HTML, CSS, and JavaScript** are used for designing the user interface, making the system more interactive and user-friendly. These technologies help in creating web-based interfaces for displaying results and interacting with the system.

Additional Tools

To improve development and execution, the following tools can also be used:

- **IDE** : PyCharm / VS Code / Jupyter Notebook
- **Libraries** : NumPy, Pandas, Scikit-learn, Matplotlib
- **Framework** : PySpark (for large data processing)

CHAPTER 11

IMPLEMENTATION

```

<!DOCTYPE html>

<html lang="en">

  <title>Login</title>

  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <head>

    <link rel="icon" href="images/icon.png" type="image/x-icon" />

    <link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">

    <style>
      body {background: url("{% static 'bg.jpg' %}");
      }
      .container-fluid {padding:50px;}
      .container{background-color:white;padding:50px; }
      #title{font-family: 'Fredoka One', cursive;
    }

      .text-uppercase{
      font-family: 'Righteous', cursive;

      }

      .style1 {color: #FF0000}
      .style4 {color: #FF0000; font-weight: bold; }
      .style5 {
      font-family: 'Righteous', cursive;
      color: #FF0000;
      font-weight: bold;
    }

    </style>
  </head>
  <body>
    <div class="container-fluid">
      <div >
        <h2 class="style1 text-center" id="title"><strong>Fraud Detection and Analysis for Insurance
Claim using Machine Learning
</strong></h2>
        <p class="text-center">Machine Learning Algorithm, PySpark, Fraud Case detection,
classifications.
        <span class="style4"><small id="passwordHelpInline" class="text-muted">

```

```

</small></span> </p>
    <hr>
    <div class="row">
        <div class="col-md-5">
            <form role="form" method="POST" >
                {% csrf_token %}
                <fieldset>
                    <p class="text-uppercase pull-center">&nbsp;</p>
                </fieldset>
            </form>
        </div>

        <div class="col-md-2">
            <!--null-->
        </div>

        <div class="col-md-5">
            <form method="POST" role="form">
                {% csrf_token %}

                <fieldset>
                    {% load static %}
                    
                    <p class="style5"> Login Using Your Account: </p>

                    <div class="form-group">
                        <input type="text" name="username" placeholder="User Name" required>
                    </div>
                    <div class="form-group">
                        <input type="password" name="password" placeholder="Password" required>
                    </div>
                    <div>
                        <input type="submit" name="submit1" class="btn btn-md" value="Login">
                    </div></br>

                    <p class="style5"> Login Using Your Account: </p>

                    <div>

                        <button class="btn btn-lg "><a href="{% url 'serviceproviderlogin' %}">SERVICE
PROVIDER</a></button>
                        <button class="btn btn-lg "><a href="{% url 'Register1'
%}">REGISTER</a></button>
                    </div>
                </fieldset>
            </form>
        </div>
    </div>

```



```

    </div>

</div>
</body>

</html>

<!DOCTYPE html>

<html lang="en">

  <title>Register Your Details</title>

  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <style type="text/css">
<!--
.style1 {
  color: #FF0000;
  font-weight: bold;
}
-->
  </style>
  <head>

    <link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">
    <link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">

    <style>
      body {background: url("{% static 'bg.jpg' %}");
      }
      .container-fluid {padding:50px;}
      .container{background-color:white;padding:50px; }
      #title{font-family: 'Fredoka One', cursive;
    }
      .text-uppercase{
        font-family: 'Righteous', cursive;

      }
      .style2 {color: #FFFF00}
    </style>
  </head>
  <body>
  <div class="container-fluid">

```

```

<div>
  <h2 class="text-center style1" id="title">Fraud Detection and Analysis for Insurance Claim using
Machine Learning
</h2>
  <p class="text-center">Machine Learning Algorithm, PySpark, Fraud Case detection,
classifications.
  <span class="style1"><small id="passwordHelpInline" class="text-muted">

</small></span> </p>
  <p class="text-center">&nbsp;</p>
  <p class="text-center">&nbsp;</p>
  <hr>
  <div class="row">
    <div class="col-md-5">
      <form role="form" method="POST" >
        {% csrf_token %}
        <fieldset>
          <p>{% load static %}
          <br>
          <span class="style1 style22">REGISTER YOUR DETAILS HERE !!!</span>    </p>
          <table width="825" border="0" align="center">
            <tr>
              <td width="184" bgcolor="#FF0000"><span class="style2 style19"><strong>Enter
Username </strong></span></td>
              <td width="199"><span class="form-group">
                <input type="text" name="username" id="username" placeholder="User Name"
required="required" />
              </span></td>
              <td width="256" bgcolor="#FF0000"><span class="style3 style22
style2"><strong>Enter Password </strong></span></td>
              <td width="168"><span class="form-group">
                <input type="password" name="password" id="password" placeholder="Password"
required="required" />
              </span></td>
            </tr>
            <tr>
              <td height="47" bgcolor="#FF0000"><span class="style2 style19"><strong>Enter
EMail Id </strong></span></td>
              <td><span class="form-group">
                <input type="email" name="email" id="email" placeholder="Enter Email "
required="required" />
              </span></td>
              <td bgcolor="#FF0000"><span class="style3 style22 style2"><strong>Enter Address
</strong></span></td>
              <td><span class="form-group">
                <textarea name="address" placeholder="Enter Address "></textarea>
              </span></td>
            </tr>
          </table>
        </fieldset>
      </form>
    </div>
  </div>

```

```

        <tr>
            <td height="37" bgcolor="#FF0000"><span class="style2 style19"><strong>Enter
Gender </strong></span></td>
            <td><select name="gender">
                <option>----Select Gender ----</option>
                <option>Male</option>
                <option>Female</option>
            </select>
            </td>
            <td bgcolor="#FF0000"><span class="style3 style22 style2"><strong>Enter Mobile
Number </strong></span></td>
            <td><span class="form-group">
                <input type="number" name="phoneno" id="phoneno" placeholder="Enter Mobile
Number" required="required" />
            </span></td>
        </tr>
        <tr>
            <td bgcolor="#FF0000"><span class="style2 style19"><strong>Enter Country Name
</strong></span></td>
            <td><span class="form-group">
                <input type="text" name="country" id="country" placeholder="Enter Country
Name" required="required" />
            </span></td>
            <td bgcolor="#FF0000"><span class="style3 style22 style2"><strong>Enter State
Name </strong></span></td>
            <td><span class="form-group">
                <input type="text" name="state" id="state" placeholder="Enter State Name"
required="required" />
            </span></td>
        </tr>
        <tr>
            <td bgcolor="#FF0000"><span class="style2 style19"><strong>Enter City Name
</strong></span></td>
            <td><span class="form-group">
                <input type="text" name="city" id="city" placeholder="Enter City Name"
required="required" />
            </span></td>
            <td bgcolor="#FF0000">&nbsp;</td>
            <td><input type="submit" class="btn btn-lg btn-primary"
name="submit" value="Register" /></td>
        </tr>
    </table>
</fieldset>
<div><br>
    <button class="btn btn-lg ">
</form>
</div>
<table width="513" border="0" align="center" >

```

```

                <tr><td width="424" bgcolor="#FF0000"><div align="center" class="style23">
                    <div align="left"><span class="style2"><strong>Registered
Status</strong></span><span class="style7 style2 style5"> ::</span> <span class="style8
style2"><strong>{{object}}</strong></span></div>
                </div></td>

```

```

</table>

```

```

<br>

```

```

        <p align="center"><span class="active"><span class="style12 style3 style10"><a href="{%
url 'login' %}">Remote User </a>|<a href="{% url 'serviceproviderlogin' %}"> Service Provider
</a></span></span></p>

```

```

        <div class="col-md-2">

```

```

            <!-------null----->

```

```

        </div>

```

```

        <div class="col-md-5">        </div>

```

```

    </div>

```

```

</div>

```

```

</div>

```

```

</body>

```

```

</html>

```

```

<link rel="icon" href="images/icon.png" type="image/x-icon" />

```

```

<link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">

```

```

<link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">

```

```

<link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">

```

```

<style>

```

```

    body {background-color:#000000;}

```

```

    .container-fluid {padding:50px;}

```

```

    .container{background-color:white;padding:50px; }

```

```

    #title{font-family: 'Fredoka One', cursive;

```

```

}

```

```

    .text-uppercase{

```

```

        font-family: 'Righteous', cursive;

```

```

    }

```

```

    .style1 {

```

```

        color: #FF0000;

```

```

        font-weight: bold;

```

```

    }

```

```

</style>

<body>
<div class="container-fluid">
  <div class="container">

    <div class="row">
      <div class="col-md-5">
        <table width="879" border="0" align="center">
          <tr>
            <td width="148" height="34" bgcolor="#FF0000"><span
class="style2"><strong>Username</strong></span></td>
            <td width="225"><span class="style9">{{object.username}}</span></td>
            <td width="157" bgcolor="#FF0000"><span class="style7 style2">Email Id
</span></td>
            <td width="321"><span class="style9">{{object.email}}</span></td>
          </tr>
          <tr>
            <td height="38" bgcolor="#FF0000"><span class="style2"><strong>Mobile
Number </strong></span></td>
            <td><span class="style9">{{object.phoneno}}</span></td>
            <td bgcolor="#FF0000"><span class="style7 style2">Gender</span></td>
            <td><span class="style9">{{object.gender}}</span></td>
          </tr>
          <tr>
            <td height="34" bgcolor="#FF0000"><span
class="style2"><strong>Address</strong></span></td>
            <td><span class="style9">{{object.address}}</span></td>
            <td bgcolor="#FF0000"><span class="style7 style2">Country</span></td>
            <td><span class="style9">{{object.country}}</span></td>
          </tr>
          <tr>
            <td height="32" bgcolor="#FF0000"><span
class="style2"><strong>State</strong></span></td>
            <td><span class="style9">{{object.state}}</span></td>
            <td bgcolor="#FF0000"><span class="style7 style2">City</span></td>
            <td><span class="style9">{{object.city}}</span></td>
          </tr>
          <tr>
            <td>&nbsp;</td>
            <td>&nbsp;</td>
            <td>&nbsp;</td>
            <td>&nbsp;</td>
          </tr>
        </table>
      </div>
    </div>
  </div>

```

```

    <div class="col-md-2">
      <!-----null----->
    </div>
  </div>
</div>
{% endblock %}

<link rel="icon" href="images/icon.png" type="image/x-icon" />

<link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">
<link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">
<link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">

<style>
  body {background-color:#000000;}
  .container-fluid {padding:50px;}
  .container{background-color:white;padding:50px; }
  #title{font-family: 'Fredoka One', cursive;
}
  .text-uppercase{
    font-family: 'Righteous', cursive;

  }
  .tweettext{

border: 2px solid yellowgreen;
width: 1104px;
height: 100px;
overflow: scroll;
background-color:;
}
  .style1 {
color: #FF0000;
font-weight: bold;
}
.style4 {color: #FFFF00; font-weight: bold; }
  .style6 {
color: #FFFF00;
font-weight: bold;
font-style: italic;
font-family: Georgia, "Times New Roman", Times, serif;
}
</style>

<body>
<div class="container-fluid">

```

```

<div class="container">

<div class="row">
  <div class="col-md-5">

    <form role="form" method="POST" >
      {% csrf_token %}
      <fieldset>
        <p class="text-uppercase pull-center style1"> PREDICTION OF INSURANCE CLAIM
TYPE!!! </p>
        <hr>

        {% csrf_token %}
        <table width="1144" border="0" align="center">

          <tr>
            <td height="44" colspan="4" bgcolor="#FF0000"><div align="center"><span
class="style6">FEED ALL INSURANCE CLAIM DETAILS HERE !!!</span> </div></td>
          </tr>
          <tr>
            <td width="314" height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter ACCOUNT_CODE</span></div></td>
            <td width="272"><div align="center">
              <input type="text" name="Account_Code">
            </div></td>
            <td width="282" bgcolor="#FF0000"><div align="center" class="style4">Enter
DATE_OF_INTIMATION</div></td>
            <td width="256"><div align="center">
              <input type="text" name="DATE_OF_INTIMATION">
            </div></td>
          </tr>
          <tr>
            <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter DATE_OF_ACCIDENT</span></div></td>
            <td><div align="center">
              <input type="text" name="DATE_OF_ACCIDENT">
            </div></td>
            <td bgcolor="#FF0000"><div align="center" class="style4">Enter
CLAIM_Real</div></td>
            <td><div align="center">
              <input type="text" name="CLAIM_Real">
            </div></td>
          </tr>
          <tr>
            <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter AGE</span></div></td>
            <td><div align="center">

```

```

        <input type="text" name="AGE">
    </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4">Enter
TYPE</div></td>
        <td><div align="center">
            <input type="text" name="TYPE">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter DRIVING_LICENSE_ISSUE</span></div></td>
        <td><div align="center">
            <input type="text" name="DRIVING_LICENSE_ISSUE">
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4">Enter
BODY_TYPE</div></td>
        <td><div align="center">
            <input type="text" name="BODY_TYPE">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter MAKE</span><span class="style4"></span> </div></td>
        <td><div align="center">
            <input type="text" name="MAKE">
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4">Enter
MODEL</div></td>
        <td><div align="center">
            <input type="text" name="MODEL">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter YEAR</span></div></td>
        <td><div align="center">
            <input type="text" name="YEAR">
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4">Enter
CHASIS_Real</div></td>
        <td><div align="center">
            <input type="text" name="CHASIS_Real">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center" class="style4">Enter
REGISTRATION COUNTRY </div></td>
        <td><div align="center">

```



```

        <input type="text" name="REG">
    </div></td>
    <td bgcolor="#FF0000"><div align="center"><span class="style4 style2">Enter
SUM_INSURED</span></div></td>
        <td><div align="center">
            <input type="text" name="SUM_INSURED">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center"><span
class="style4">Enter POLICY_NO</span></div></td>
        <td><div align="center">
            <input type="text" name="POLICY_NO" >
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4"><span
class="style2">Enter</span> POLICY_START</div></td>
        <td><div align="center">
            <input type="text" name="POLICY_START">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center" class="style4">
            <div align="center">Enter POLICY_END</div>
        </div></td>
        <td><div align="center">
            <input type="text" name="POLICY_END" >
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4"><span
class="style2">Enter</span> INTIMATED_AMOUNT</div></td>
        <td><div align="center">
            <input type="text" name="INTIMATED_AMOUNT">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center" class="style4">
            <div align="center">Enter INTIMATED_SF</div>
        </div></td>
        <td><div align="center">
            <input type="text" name="INTIMATED_SF" >
        </div></td>
        <td bgcolor="#FF0000"><div align="center" class="style4"><span
class="style2">Enter</span> EXECUTIVE</div></td>
        <td><div align="center">
            <input type="text" name="EXECUTIVE">
        </div></td>
    </tr>
    <tr>
        <td height="44" bgcolor="#FF0000"><div align="center" class="style4">

```

```

        <div align="center">Enter PRODUCT</div>
    </div></td>
    <td><div align="center">
        <input type="text" name="PRODUCT" >
    </div></td>
    <td bgcolor="#FF0000"><div align="center" class="style4">Enter POLICY
TYPE</div></td>
    <td><div align="center">
        <input type="text" name="POLICYTYPE">
    </div></td>
</tr>

<tr>
    <td height="44" bgcolor="#FF0000"><div align="center" class="style4">
        <div align="center" class="style4">Enter NATIONALITY</div>
    </div></td>
    <td bgcolor="#FFFFFF"><div align="center">
        <input type="text" name="NATIONALITY" >
    </div></td>
    <td bgcolor="#FF0000">&nbsp;</td>
    <td bgcolor="#FFFFFF">&nbsp;</td>
</tr>
<tr>
    <td height="44" bgcolor="#FF0000">&nbsp;</td>
    <td bgcolor="#FF0000">&nbsp;</td>
    <td bgcolor="#FF0000"><input name="submit" type="submit" class="style1"
value="Predict"></td>
    <td bgcolor="#FF0000">&nbsp;</td>
</tr>
<td><p>&nbsp;</p>    </td>
</tr>
</table>
</fieldset>
</form>

<form role="form" method="POST" >
    {% csrf_token %}
    <fieldset>

        <hr>
        <div>
            <table border="5" align="center" bordercolor="#FF00FF">
                <tr><td width="328" bgcolor="#FF0000"><div align="center"><span
class="style4"><strong>PREDICTED        INSURANCE        CLAIM        TYPE        ::
</strong></span></div></td>

```

```
<td width="275" height="68" bgcolor="#FFFFFF" style="color:red; font-size:20px; font-family:fantasy" ><div align="center">{{objs}}</div></td>
```

```
</tr>
```

```
</table>
```

```
</div>
```

```
</fieldset>
```

```
</form>
```

```
</div>
```

```
<div class="col-md-2">
```

```
<!--null-->
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
{% endblock %}
```

```
<tr>
```

```
from django.db.models import Count, Avg
from django.shortcuts import render, redirect
from django.db.models import Count
from django.db.models import Q
import datetime
import xlwt
from django.http import HttpResponse
```

```
import pandas as pd
```

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
# Create your views here.
FromRemote_User.modelsimport
ClientRegister_Model,insurance_claim_status,detection_accuracy,detection_ratio
```

```
def serviceproviderlogin(request):
    if request.method == "POST":
        admin = request.POST.get('username')
        password = request.POST.get('password')
        if admin == "Admin" and password=="Admin":
```

```

        return redirect('View_Remote_Users')

    return render(request, 'Service_Provider/serviceproviderlogin.html')

def Find_Insurance_Claim_Prediction_Details_Ratio(request):
    detection_ratio.objects.all().delete()
    ratio = ""
    kword = 'Fraud Claim'
    print(kword)
    obj = insurance_claim_status.objects.all().filter(Q(PREDICTION=kword))
    obj1 = insurance_claim_status.objects.all()
    count = obj.count();
    count1 = obj1.count();
    ratio = (count / count1) * 100
    if ratio != 0:
        detection_ratio.objects.create(names=kword, ratio=ratio)

    ratio1 = ""
    kword1 = 'Real Claim'
    print(kword1)
    obj1 = insurance_claim_status.objects.all().filter(Q(PREDICTION=kword1))
    obj11 = insurance_claim_status.objects.all()
    count1 = obj1.count();
    count11 = obj11.count();
    ratio1 = (count1 / count11) * 100
    if ratio1 != 0:
        detection_ratio.objects.create(names=kword1, ratio=ratio1)

    obj = detection_ratio.objects.all()

    return render(request, 'Service_Provider/Find_Insurance_Claim_Prediction_Details_Ratio.html',
{'objs': obj})

def View_Insurance_Claim_Prediction_Details(request):

    obj = insurance_claim_status.objects.all()
    return render(request, 'Service_Provider/View_Insurance_Claim_Prediction_Details.html', {'objs':
obj})

def View_Remote_Users(request):
    obj=ClientRegister_Model.objects.all()
    return render(request, 'Service_Provider/View_Remote_Users.html', {'objects':obj})

def ViewTrendings(request):
    topic = detection_accuracy.objects.values('topics').annotate(dcount=Count('topics')).order_by('-
dcount')
    return render(request, 'Service_Provider/ViewTrendings.html', {'objects':topic})

```

```

def charts(request,chart_type):
    chart1 = detection_ratio.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"Service_Provider/charts.html", {'form':chart1, 'chart_type':chart_type})

def charts1(request,chart_type):
    chart1 = detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"Service_Provider/charts1.html", {'form':chart1, 'chart_type':chart_type})

def Train_Test_View_Results_Details(request):
    detection_accuracy.objects.all().delete()
    data = pd.read_csv("Insurance_Claim_Datasets.csv", encoding='latin-1')

    def apply_results(results):
        if (results == 'Fraud'):
            return 0
        elif (results == 'Real'):
            return 1

    data['Results'] = data['Claim_Staus'].apply(apply_results)

    x = data['POLICY_NO'].apply(str)
    y = data['Results']

    # data.drop(['Type_of_Breach'],axis = 1, inplace = True)
    cv = CountVectorizer()

    print(x)
    print(y)

    cv = CountVectorizer(lowercase=False, strip_accents='unicode', ngram_range=(1, 1))
    # x = cv.fit_transform(data['POLICY_NO'].apply(lambda x: np.str_(x)))
    x = cv.fit_transform(x)

    models = []
    from sklearn.model_selection import train_test_split
    X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.20)
    X_train.shape, X_test.shape, y_train.shape

    print("Naive Bayes")

    from sklearn.naive_bayes import MultinomialNB

    NB = MultinomialNB()
    NB.fit(X_train, y_train)
    predict_nb = NB.predict(X_test)
    naivebayes = accuracy_score(y_test, predict_nb) * 180
    print("ACCURACY")

```

```

print(naivebayes)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_nb))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_nb))
detection_accuracy.objects.create(names="Naive Bayes", ratio=naivebayes)

# SVM Model
print("SVM")
from sklearn import svm

lin_clf = svm.LinearSVC()
lin_clf.fit(X_train, y_train)
predict_svm = lin_clf.predict(X_test)
svm_acc = accuracy_score(y_test, predict_svm) * 180
print("ACCURACY")
print(svm_acc)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, predict_svm))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, predict_svm))
detection_accuracy.objects.create(names="SVM", ratio=svm_acc)

print("Logistic Regression")

from sklearn.linear_model import LogisticRegression

reg = LogisticRegression(random_state=0, solver='lbfgs').fit(X_train, y_train)
y_pred = reg.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, y_pred) * 180)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, y_pred))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
detection_accuracy.objects.create(names="Logistic Regression", ratio=accuracy_score(y_test,
y_pred) * 180)

print("Decision Tree Classifier")
dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)
dtcpredict = dtc.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, dtcpredict) * 180)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, dtcpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, dtcpredict))

```

```
detection_accuracy.objects.create(names="Decision Tree Classifier", ratio=accuracy_score(y_test,
dtcpredict) * 180)
```

```
print("SGD Classifier")
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(loss='hinge', penalty='l2', random_state=0)
sgd_clf.fit(X_train, y_train)
sgdpredict = sgd_clf.predict(X_test)
print("ACCURACY")
print(accuracy_score(y_test, sgdpredict) * 180)
print("CLASSIFICATION REPORT")
print(classification_report(y_test, sgdpredict))
print("CONFUSION MATRIX")
print(confusion_matrix(y_test, sgdpredict))
detection_accuracy.objects.create(names="SGD Classifier", ratio=accuracy_score(y_test,
sgdpredict) * 180)
```

```
labeled = 'labeled_data.csv'
data.to_csv(labeled, index=False)
data.to_markdown
```

```
obj =detection_accuracy.objects.all()
return render(request, 'Service_Provider/Train_Test_View_Results_Details.html', {'objs': obj})
```

```
def likeschart(request,like_chart):
    charts =detection_accuracy.objects.values('names').annotate(dcount=Avg('ratio'))
    return render(request,"Service_Provider/likeschart.html", {'form':charts, 'like_chart':like_chart})
```

```
def Download_Trained_DataSets(request):
```

```
    response = HttpResponse(content_type='application/ms-excel')
    # decide file name
    response['Content-Disposition'] = 'attachment; filename="Predicted_Data.xls"'
    # creating workbook
    wb = xlwt.Workbook(encoding='utf-8')
    # adding sheet
    ws = wb.add_sheet("sheet1")
    # Sheet header, first row
    row_num = 0
    font_style = xlwt.XFStyle()
    # headers are bold
    font_style.font.bold = True
    # writer = csv.writer(response)
    obj = insurance_claim_status.objects.all()
    data = obj # dummy method to fetch data.
    for my_row in data:
        row_num = row_num + 1
```

```
ws.write(row_num, 0, my_row.Account_Code, font_style)
ws.write(row_num, 1, my_row.DATE_OF_INTIMATION, font_style)
ws.write(row_num, 2, my_row.DATE_OF_ACCIDENT, font_style)
ws.write(row_num, 3, my_row.CLAIM_Real, font_style)
ws.write(row_num, 4, my_row.AGE, font_style)
ws.write(row_num, 5, my_row.TYPE, font_style)
ws.write(row_num, 6, my_row.DRIVING_LICENSE_ISSUE, font_style)
ws.write(row_num, 7, my_row.BODY_TYPE, font_style)
ws.write(row_num, 8, my_row.MAKE, font_style)
ws.write(row_num, 9, my_row.MODEL, font_style)
ws.write(row_num, 10, my_row.YEAR, font_style)
ws.write(row_num, 11, my_row.CHASIS_Real, font_style)
ws.write(row_num, 12, my_row.REG, font_style)
ws.write(row_num, 13, my_row.SUM_INSURED, font_style)
ws.write(row_num, 14, my_row.POLICY_NO, font_style)
ws.write(row_num, 15, my_row.POLICY_START, font_style)
ws.write(row_num, 16, my_row.POLICY_END, font_style)
ws.write(row_num, 17, my_row.INTIMATED_AMOUNT, font_style)
ws.write(row_num, 18, my_row.INTIMATED_SF, font_style)
ws.write(row_num, 19, my_row.EXECUTIVE, font_style)
ws.write(row_num, 20, my_row.PRODUCT, font_style)
ws.write(row_num, 21, my_row.POLICYTYPE, font_style)
ws.write(row_num, 22, my_row.NATIONALITY, font_style)
ws.write(row_num, 23, my_row.PREDICTION, font_style)
```

```
wb.save(response)
return response
```


CHAPTER 11

UML DIAGRAMS

UML DIAGRAMS

➤ USECASE DIAGRAM

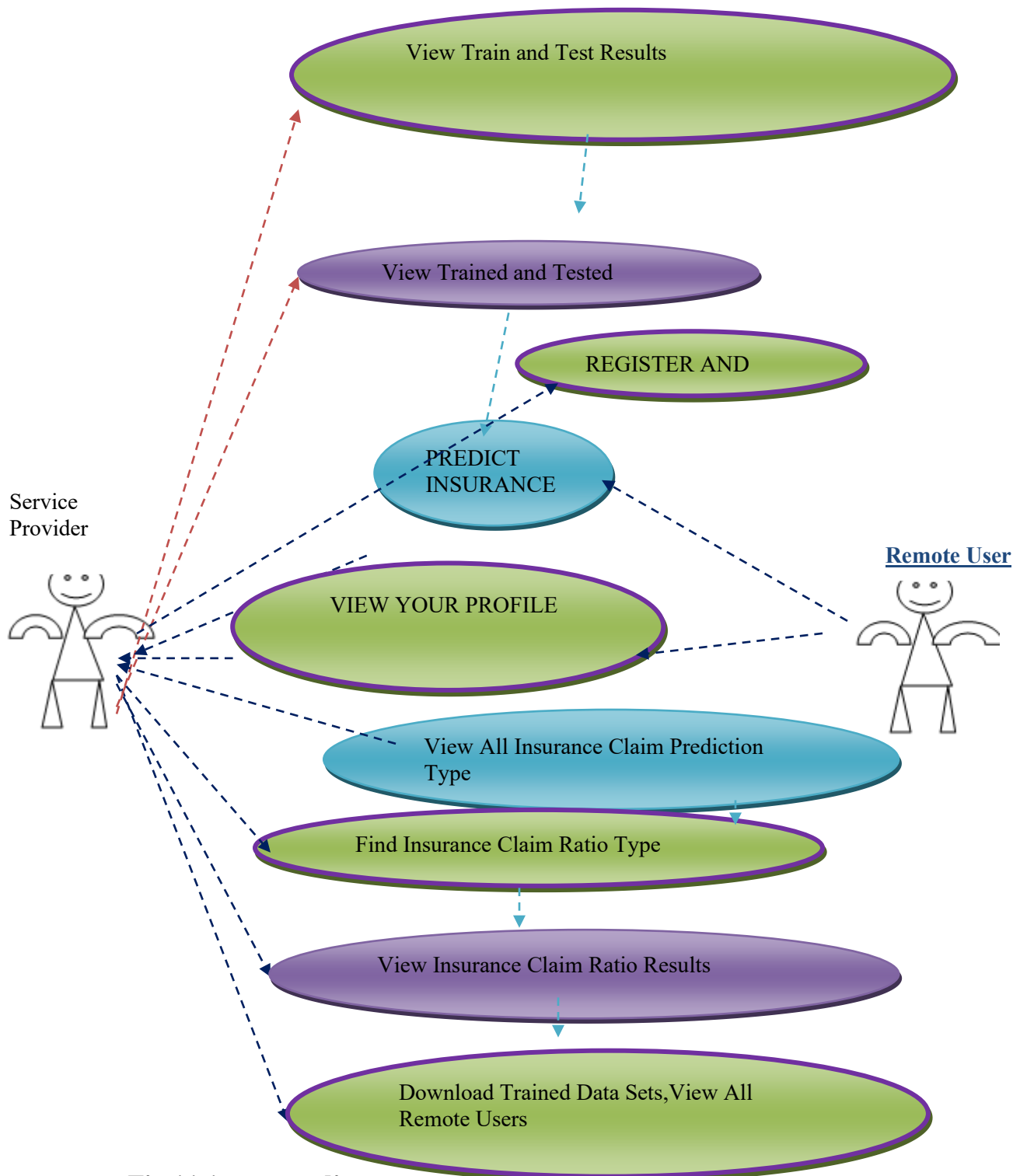


Fig 11.1 usecase diagram

A Use Case Diagram is used to show how users interact with the system. It explains what actions users can perform and how the system responds. It gives a simple overview of the system functions.

In this project, the fraud detection system has two main users: Admin and User (Insurance Staff). The admin manages the system, while the user checks insurance claims.

The admin can log in, upload the dataset, train the model, and view reports. The user can log in, enter claim details, and check whether the claim is fraudulent or not. The system processes the data and shows the result.

The use case diagram helps to understand the system easily. It shows all important functions like login, data processing, fraud detection, and result display in a simple visual form.

CLASSS DIAGRAM

A Class Diagram is used to represent the structure of a system by showing its classes, attributes, and relationships. It is one of the important diagrams in system design that helps in understanding how different parts of the system are organized.

In the fraud detection system, the class diagram shows the main classes such as User, Admin, Dataset, Model, and Prediction. Each class contains attributes (data) and methods (functions). For example, the User class may contain details like username and password, while the Model class contains functions for training and prediction.

The Admin class manages the system by uploading datasets and viewing results. The Dataset class stores insurance claim data. The Model class is responsible for training the machine learning algorithm. The Prediction class handles the output and shows whether a claim is fraudulent or not.

These classes are connected with relationships such as association, which shows how one class interacts with another. For example, the User interacts with the Prediction class to check results, and the Admin interacts with the Dataset and Model classes.

Service Provider

Methods

Members

Login, Train and Test Data Sets, View Train and Test Results, View Trained and Tested Results By Bar Chart, View All Insurance Claim Prediction Type, Find Insurance Claim Ratio Type, View Insurance Claim Ratio Results, Download Trained Data Sets, View All Remote Users. Account_Code, DATE_OF_INTIMATION, DATE_OF_ACCIDENT, CLAIM_Real, AGE, TYPE, DRIVING_LICENSE_ISSUE, BODY_TYPE, MAKE, MODEL, YEAR, CHASIS_Real, REG, SUM_INSURED, POLICY_NO, POLICY_START, POLICY_END, INTIMATED_AMOUNT, INTIMATED_SF, EXECUTIVE, PRODUCT, POLICYTYPE, NATIONALITY, PREDICTION.

Login

Methods

Members

Login (), Reset (), Register ().
User Name, Password.

Register

Methods

Members

Register (), Reset ()
User Name, Password, E-mail, Mobile, Address, DOB, Gender, Pin code, Image

Methods

Members

REGISTER AND LOGIN, PREDICT INSURANCE CLAIM TYPE, VIEW YOUR PROFILE.

Account_Code, DATE_OF_INTIMATION, DATE_OF_ACCIDENT, CLAIM_Real, AGE, TYPE, DRIVING_LICENSE_ISSUE, BODY_TYPE, MAKE, MODEL, YEAR, CHASIS_Real, REG, SUM_INSURED, POLICY_NO, POLICY_START, POLICY_END, INTIMATED_AMOUNT, INTIMATED_SF, EXECUTIVE, PRODUCT, POLICYTYPE, NATIONALITY, PREDICTION.

Remote User

Fig11.2 class diagram

SEQUENCE DIAGRAM

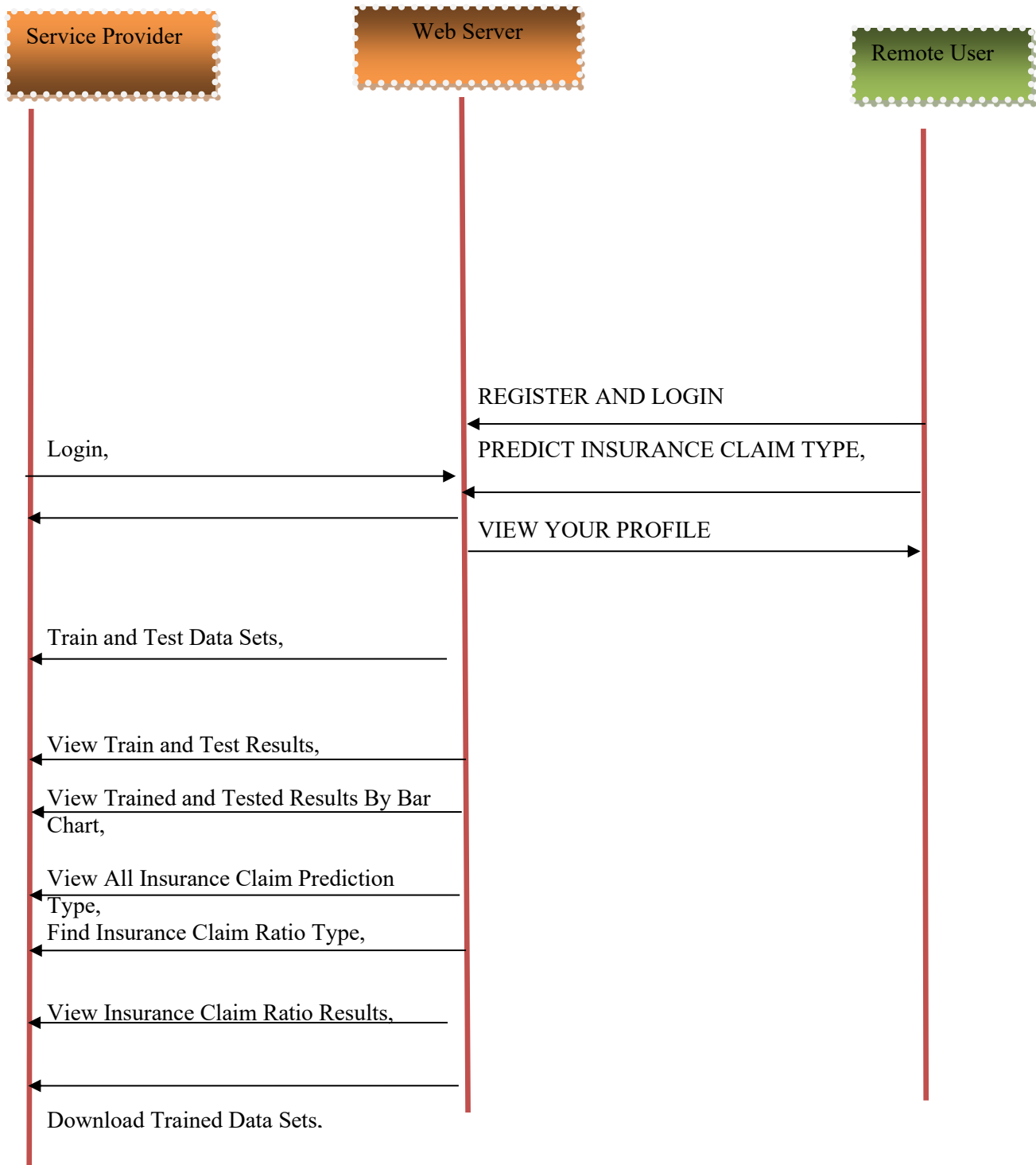


Fig 11.3 sequence diagram

A Sequence Diagram is used to show how different parts of a system interact with each other in a sequence of steps. It represents the flow of messages between users (actors) and system components over time.

In the fraud detection system, the sequence diagram shows how a user interacts with the system to check whether an insurance claim is fraudulent or not. First, the user logs into the system. Then, the user enters the claim details. The system sends this data to the processing module, where preprocessing and model prediction take place. The machine learning model analyzes the data and sends the result back to the system. Finally, the system displays the output as “Fraud” or “Not Fraud” to the user.

Similarly, the admin can log in, upload the dataset, and train the model. The system processes the dataset and updates the model for future predictions.

The sequence diagram clearly shows the order of operations and how data flows between different components step by step. It helps in understanding the dynamic behavior of the system.

DATAFLOW DIAGRAM

A Data Flow Diagram (DFD) is a graphical representation used to show how data flows through a system. It illustrates the movement of data from input to processing and finally to output. The DFD helps in understanding how the system processes information and how different components interact with each other.

In the fraud detection system, the Data Flow Diagram shows how insurance claim data is entered by the user, processed by the system, and then classified as fraudulent or non-fraudulent. It clearly represents the flow of data between users, system processes, and data storage.

At the initial stage, the user provides input in the form of insurance claim details. This data is then sent to the preprocessing module, where it is cleaned and prepared for analysis. After preprocessing, the data is passed to the machine learning model, which analyzes the input and

predicts whether the claim is fraudulent or genuine. The result is then displayed to the user. The DFD also includes data storage components such as the dataset and database, where information is stored and retrieved when needed. It shows how the admin can upload datasets and how the system uses this data for training the model.

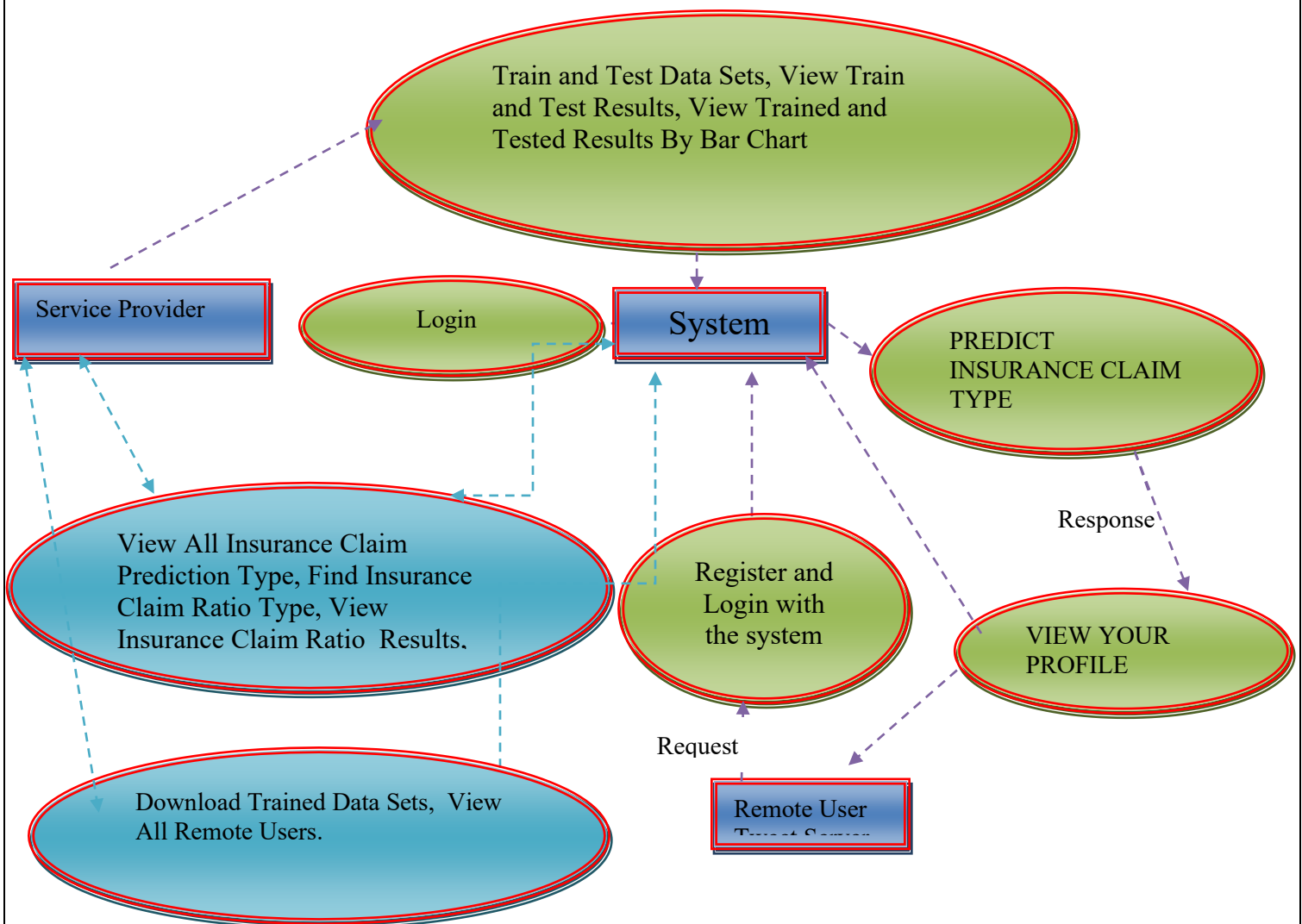
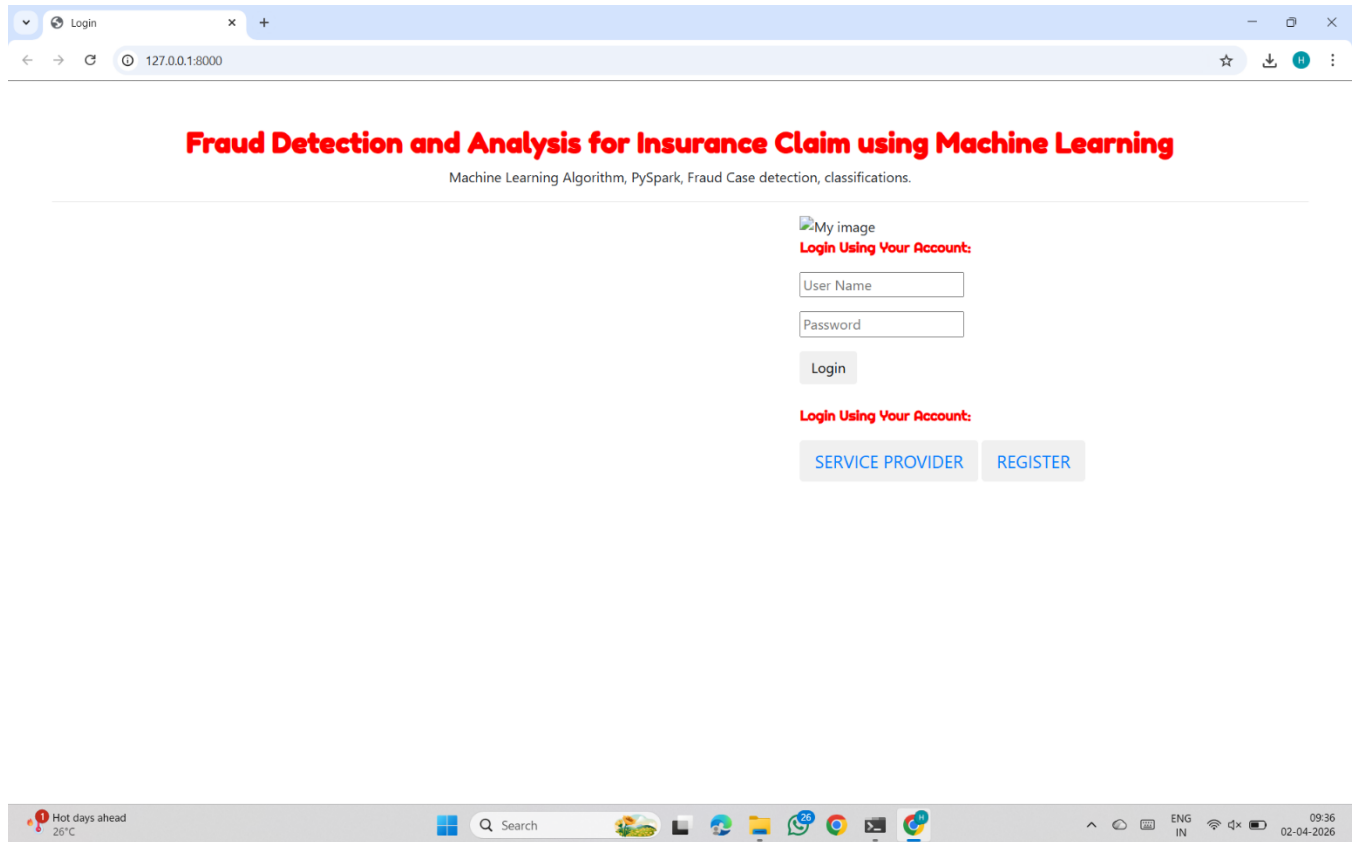


Fig 11.4 Dataflow diagram

CHAPTER 12

OUTPUTS

OUTPUTS



Login Page

The above screen represents the Login Page of the Fraud Detection and Analysis for Insurance Claim system. This is the initial interface where users can access the system securely.

The page displays the project title at the top, indicating the purpose of the system, which is fraud detection using machine learning techniques. Below the title, a login form is provided for users to enter their credentials.

The user is required to enter:

- Username
- Password

After entering valid details, the user can click on the Login button to access the system. If the credentials are correct, the user will be redirected to the main dashboard.

Additionally, the page provides options such as:

- Service Provider Login (for admin or authorized personnel)
- Register (for new users to create an account)

Register Your Details

127.0.0.1:8000/Register1/

Fraud Detection and Analysis for Insurance Claim using Machine Learning

Machine Learning Algorithm, PySpark, Fraud Case detection, classifications.

My image

REGISTER YOUR DETAILS HERE !!!

Enter Username	User Name	Enter Password	Password
Enter EMail Id	Enter Email	Enter Address	Enter Address
Enter Gender	---Select Gender ---	Enter Mobile Number	Enter Mobile Number
Enter Country Name	Enter Country Name	Enter State Name	Enter State Name
Enter City Name	Enter City Name	Register	

Registered Status ::

Remote User | Service Provider

26°C Sunny

Search

ENG IN

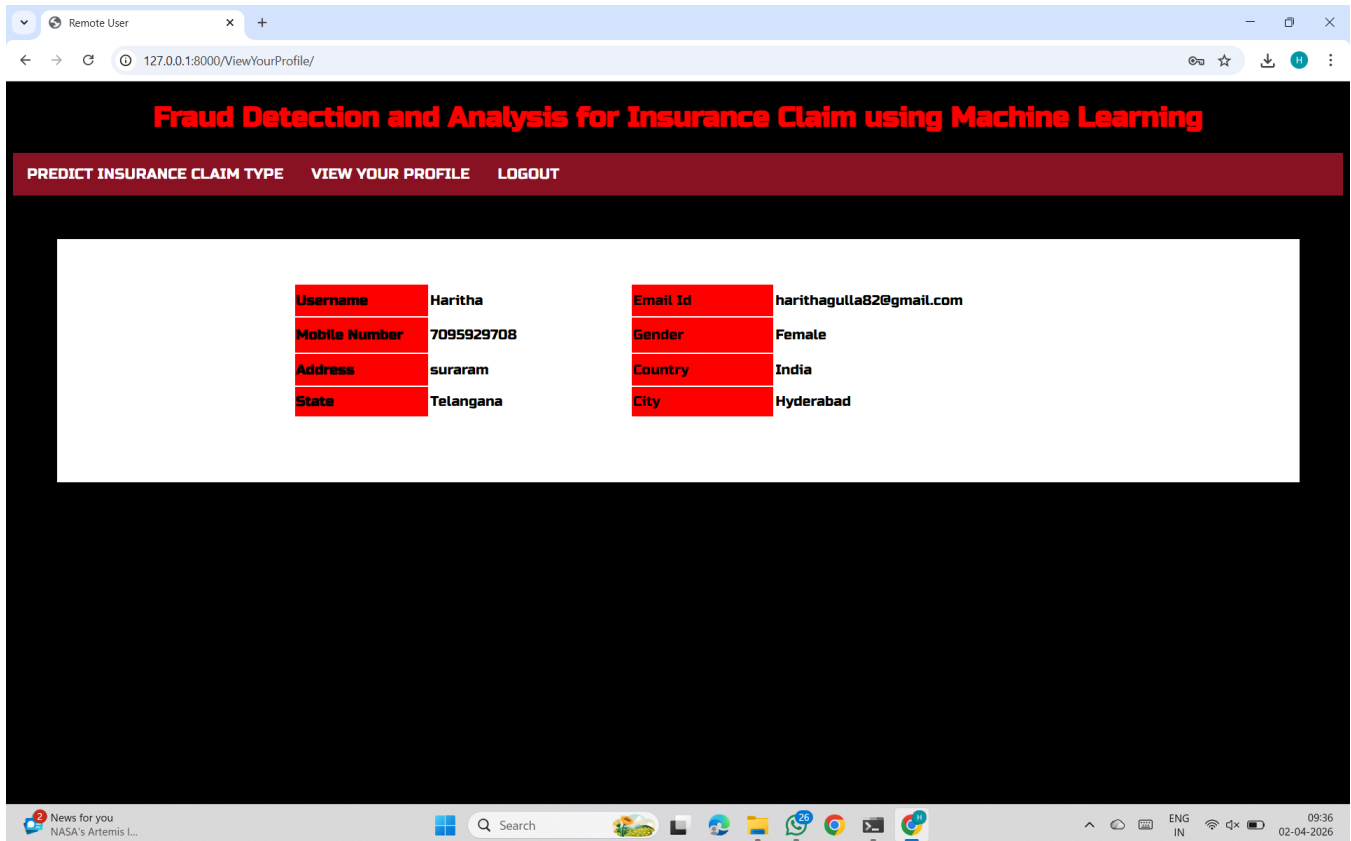
09:39 02-04-2026

Register Your Details Page

The “Register Your Details” page is used for **new users to create an account** in the fraud detection system. This page appears after selecting the register option from the login page.

In this page, the user is required to enter personal and login details such as:

- Username
- Password
- Email ID
- Mobile Number
- Gender
- Address
- Country, State, City



User Profile Page

The above screen represents the **User Profile Page** of the Fraud Detection and Analysis for Insurance Claim system. This page is displayed after the user successfully logs into the system.

The page shows the **personal details of the user**, which helps in verifying and managing user information. It includes details such as:

- Username
- Email ID
- Mobile Number
- Gender
- Address
- Country
- State
- City

FEED ALL INSURANCE CLAIM DETAILS HERE !!!

Enter ACCOUNT_CODE	300	Enter DATE_OF_INTIMATION	0
Enter DATE_OF_ACCIDENT	31-10-2016 20:03	Enter CLAIM_Real	real
Enter AGE	20	Enter TYPE	OD Claim
Enter DRIVING_LICENSE_ISSUE	27-04-2004	Enter BODY_TYPE	PICK UP
Enter MAKE	TOYOTA	Enter MODEL	golf
Enter YEAR	2013	Enter CHASIS_Real	JTEJU3FJ9E5061069
Enter REGISTRATION_COUNTRY	India	Enter SUM_INSURED	0
Enter POLICY_NO	0	Enter POLICY_START	15-09-2016
Enter POLICY_END	24-09-2017	Enter INTIMATED_AMOUNT	0
Enter INTIMATED_SF	200	Enter EXECUTIVE	JOHN
Enter PRODUCT	LUXURY	Enter POLICY TYPE	TP
Enter NATIONALITY	indian		

Predict

PREDICTED INSURANCE CLAIM TYPE :

Insurance Claim Prediction Page

The above screen represents the **Insurance Claim Prediction Page** of the system. This page is used to enter insurance claim details and predict whether the claim is fraudulent or genuine.

In this screen, the user is required to provide various details related to the insurance claim, such as:

- Account Code
- Date of Accident
- Age
- Driving License Issue Date
- Vehicle Make and Model
- Registration Country
- Policy Details (Start, End, Policy Type)
- Claim Amount and Insured Amount

Remote User x +

127.0.0.1:8000/Predict_Insurance_Claim_Type/

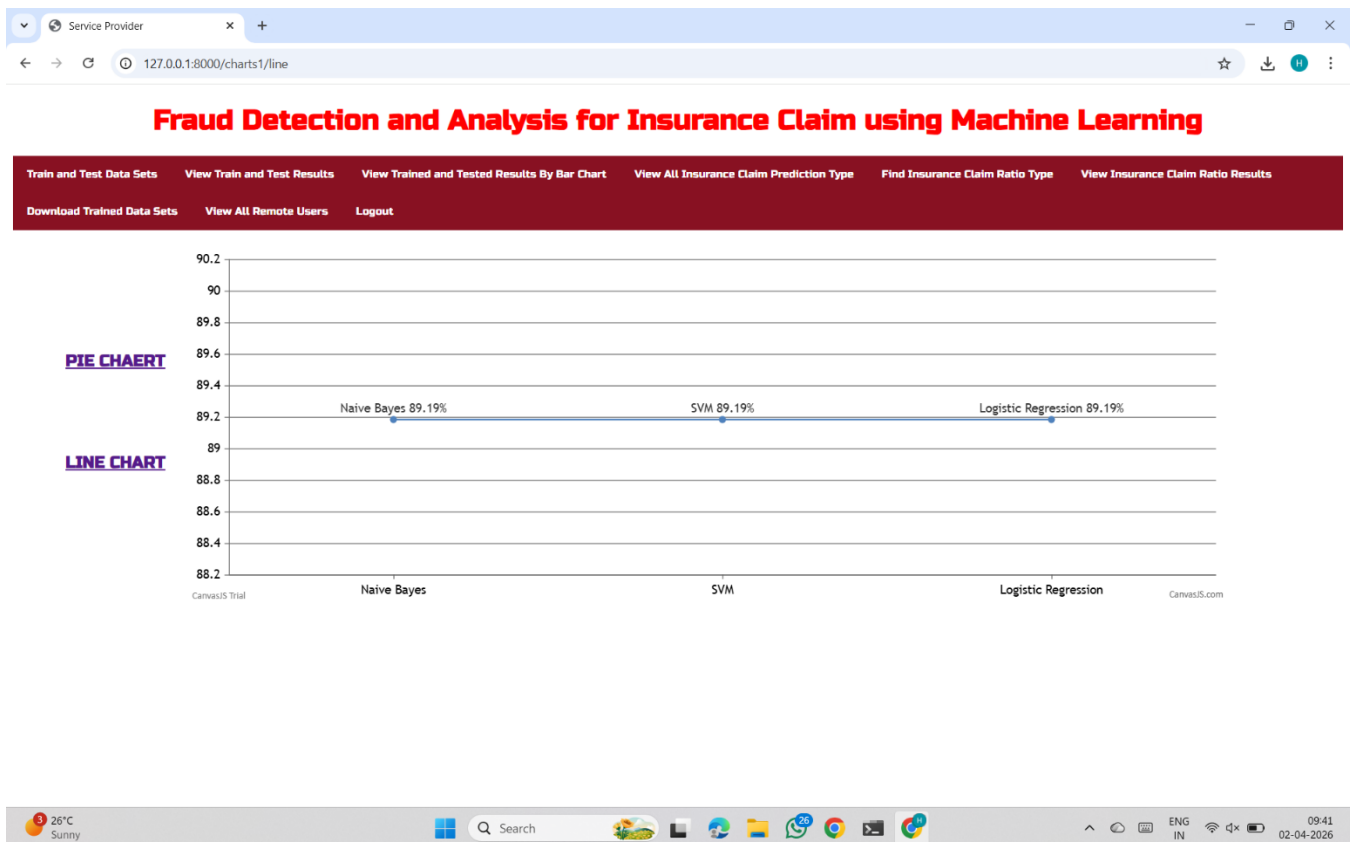
Enter AGE		Enter TYPE	
Enter DRIVING_LICENSE_ISSUE		Enter BODY_TYPE	
Enter MAKE		Enter MODEL	
Enter YEAR		Enter CHASIS_Real	
Enter REGISTRATION_COUNTRY		Enter SUM_INSURED	
Enter POLICY_NO		Enter POLICY_START	
Enter POLICY_END		Enter INTIMATED_AMOUNT	
Enter INTIMATED_SF		Enter EXECUTIVE	
Enter PRODUCT		Enter POLICY TYPE	
Enter NATIONALITY			
		Predict	

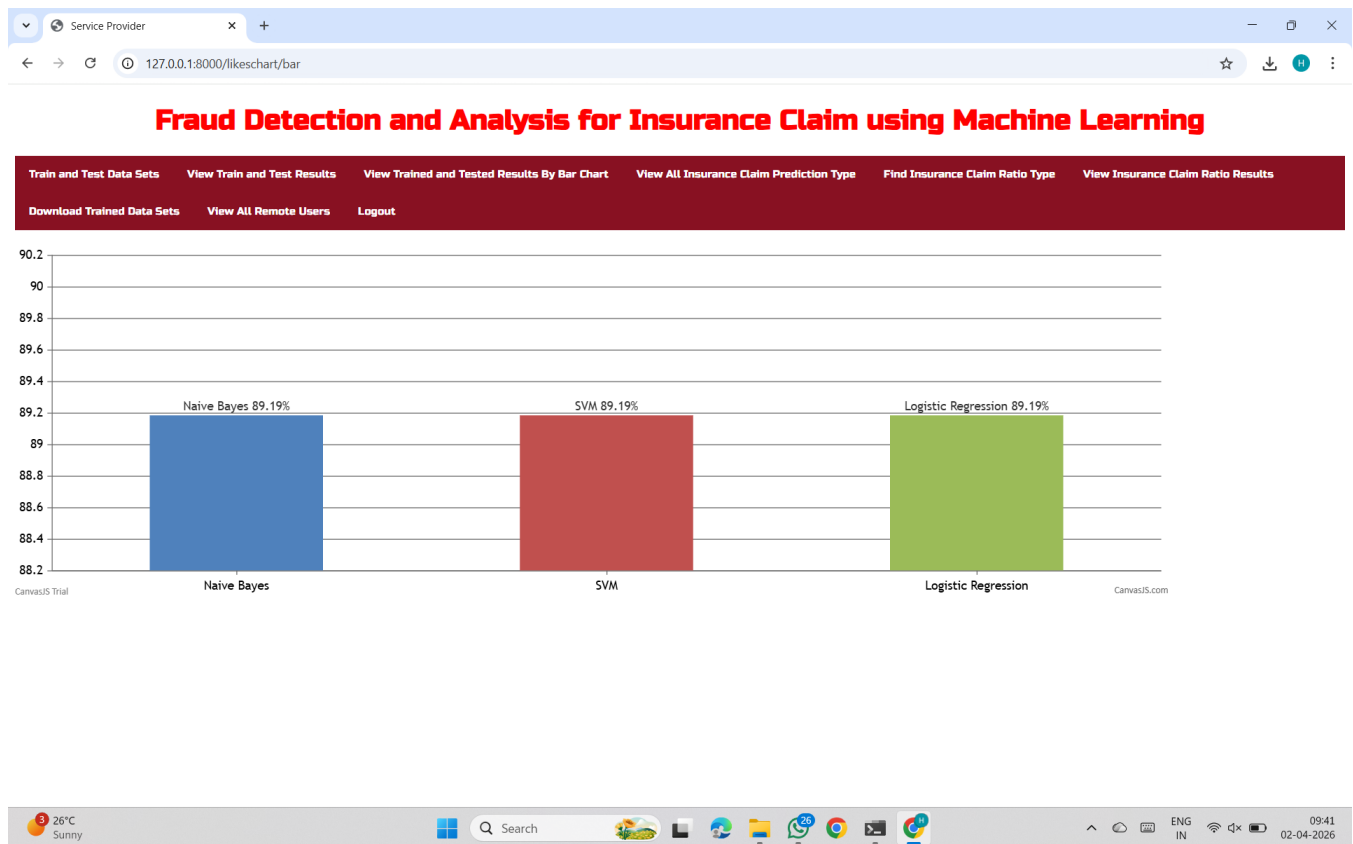
PREDICTED INSURANCE CLAIM TYPE = **Fraud Claim**

News for you NASA's Artemis L...

Search

ENG IN 09:38 02-04-2026

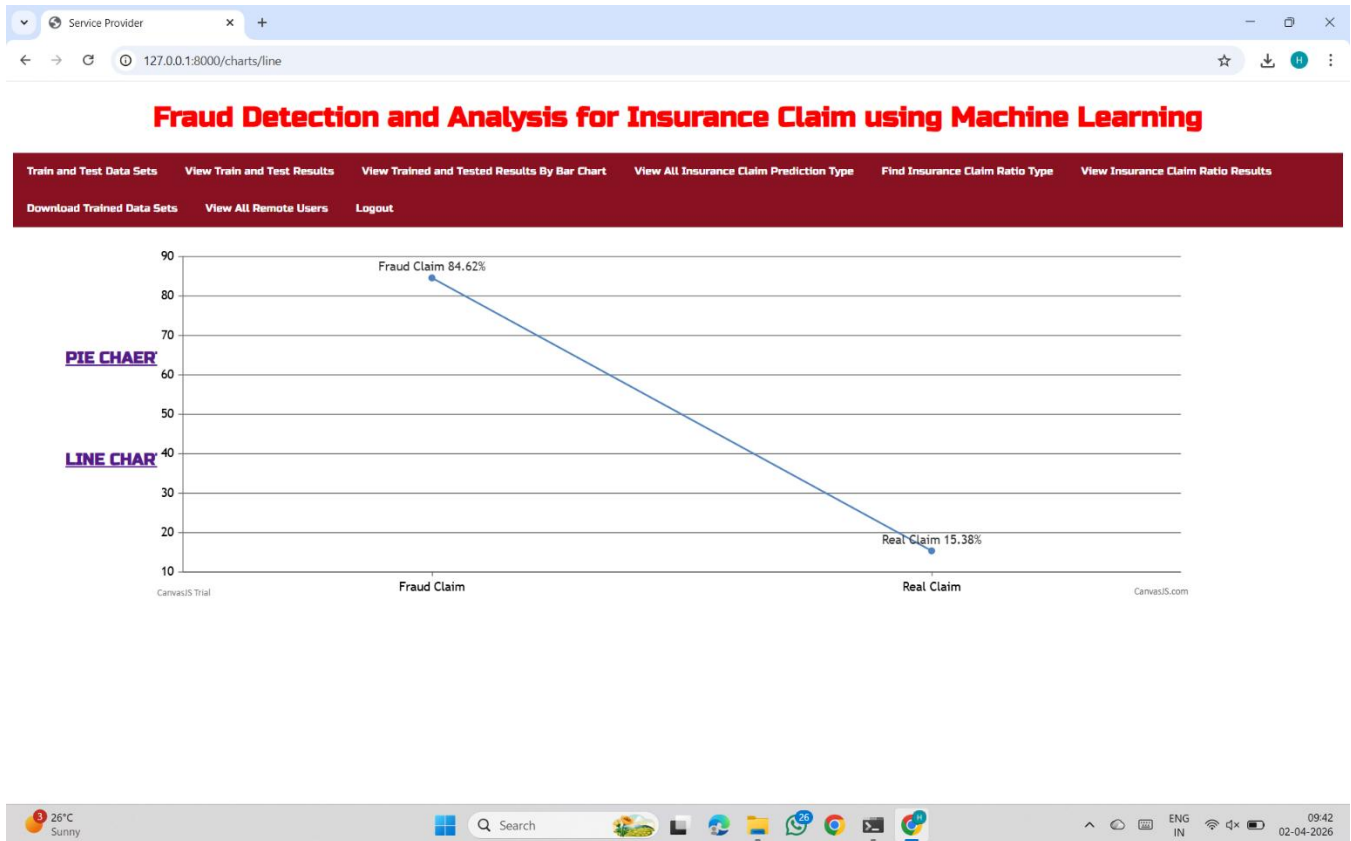




The graph in this project represents the **performance comparison of different machine learning algorithms** such as **Naive Bayes, Support Vector Machine (SVM), and Logistic Regression** used for fraud detection in insurance claims.

This graph is usually plotted using performance metrics like **accuracy, precision, recall, or F1-score** on the Y-axis, while the X-axis represents the different algorithms. Each algorithm is shown with its corresponding performance value, making it easy to compare their effectiveness.

From the graph, we can observe how each algorithm performs on the given dataset. For example, Logistic Regression may provide stable and consistent results, while SVM may achieve higher accuracy in some cases due to its ability to handle complex data patterns. Naive Bayes, being a simple probabilistic model, may perform faster but sometimes with slightly lower accuracy compared to other algorithms.



A Pie Chart is used in this project to represent the distribution of insurance claims in a visual and easy-to-understand format. It shows the proportion of fraudulent claims and non-fraudulent (genuine) claims in the dataset.

In the fraud detection system, the pie chart divides the data into sections based on categories such as Fraud and Not Fraud. Each section represents the percentage of claims belonging to that category. For example, a larger portion of the chart may represent genuine claims, while a smaller portion represents fraudulent claims, indicating that fraud cases are fewer in number.

This visualization helps in understanding the imbalance in the dataset, which is an important factor in machine learning. Since fraud cases are usually less compared to genuine ones, the pie chart clearly highlights this difference.

CHAPTER 13

BENEFITS

AND LIMITATIONS

BENEFITS AND LIMITATIONS

Benefits

The proposed fraud detection system provides several advantages to insurance companies and users by improving the efficiency and accuracy of claim verification.

One of the main benefits is automation. The system reduces the need for manual verification by automatically analyzing insurance claims using machine learning algorithms. This saves time and reduces human effort.

Another important benefit is improved accuracy. The system uses trained models to detect fraudulent claims based on patterns in data. This helps in identifying fraud more accurately compared to traditional rule-based methods.

The system also supports fast processing. It can analyze large amounts of data quickly and provide instant results, which helps in faster decision-making.

In addition, the project handles complex and hidden patterns in data. Machine learning models can detect fraud patterns that are not easily visible to humans.

The system is also scalable, meaning it can handle large datasets and can be extended for real-world applications. By using tools like PySpark, the system can process big data efficiently.

Another advantage is reduction in financial losses. By detecting fraudulent claims early, insurance companies can avoid unnecessary payouts and improve profitability.

The system also provides data-driven decision making, which improves reliability and consistency in fraud detection.

The system improves consistency in decision-making. Unlike manual verification, where decisions may vary from person to person, the machine learning model provides uniform results based on data patterns.

It also helps in early fraud detection, which is very important for preventing financial loss. Suspicious claims can be identified at an early stage and investigated further.

Another benefit is time efficiency. The system reduces the processing time required for claim verification from hours or days to just a few seconds.

The project also supports easy integration with existing insurance systems. It can be connected with company databases and web applications to enhance current workflows.

The system provides better customer service. Genuine claims can be processed faster, improving customer satisfaction and trust in the insurance company.

It also allows continuous learning. As new data is added, the model can be retrained to improve its performance and adapt to new fraud patterns.

The system can also be used for data analysis and reporting, helping companies understand trends and patterns in fraudulent activities.

Limitations

Despite its advantages, the system also has some limitations.

One major limitation is dependence on data quality. If the dataset contains incorrect, incomplete, or biased data, the model may produce inaccurate results.

Another limitation is the imbalance in data. Fraud cases are usually fewer compared to genuine claims, which may affect the model's ability to detect fraud effectively.

The system may also face issues with new or unknown fraud patterns. If a fraud type is not present in the training data, the model may not be able to detect it.

Machine learning models require training time and computational resources, especially for large datasets. This can increase system complexity.

Another limitation is lack of interpretability. Some machine learning models act as black boxes, making it difficult to understand how decisions are made.

The system also requires regular updates and maintenance. The model must be retrained with new data to maintain accuracy over time.

One limitation is overfitting, where the model performs well on training data but may not perform equally well on new or unseen data.

The system may also generate false positives, where genuine claims are incorrectly classified as fraudulent, which can affect customer trust.

Similarly, false negatives can occur, where fraudulent claims are not detected, leading to financial loss.

Another limitation is dependency on technical knowledge. Proper understanding of machine learning and data handling is required to develop and maintain the system.

CHAPTER 14

CONCLUSION

Conclusion

The project “Fraud Detection and Analysis of Insurance Claims using Machine Learning” focuses on identifying fraudulent insurance claims using advanced data-driven techniques. With the increasing number of fraud cases in the insurance industry, there is a strong need for an efficient and automated system that can accurately detect suspicious activities. This project successfully addresses this problem by applying machine learning algorithms to analyze insurance claim data and classify them as fraudulent or genuine.

The system uses various steps such as data preprocessing, feature engineering, model training, and evaluation to ensure accurate predictions. Machine learning algorithms like Logistic Regression, Decision Tree, and Random Forest are implemented and compared to select the best-performing model. The use of evaluation metrics such as accuracy, precision, recall, and confusion matrix helps in measuring the effectiveness of the system.

One of the key achievements of this project is the ability to handle large datasets and detect hidden patterns that are difficult to identify using traditional methods. The integration of tools like PySpark further improves the scalability and performance of the system, making it suitable for real-world applications. The system also reduces manual effort and speeds up the fraud detection process, leading to better decision-making.

The developed application provides a user-friendly interface where users can enter insurance claim details and instantly receive predictions. This makes the system practical and easy to use for insurance companies. By automating the fraud detection process, the system helps in reducing financial losses and improving operational efficiency.

Although the system has some limitations such as dependency on data quality and difficulty in detecting completely new fraud patterns, it still provides a strong foundation for intelligent fraud detection. With continuous updates and improvements, the system can become even more accurate and reliable.

In conclusion, this project demonstrates how machine learning can be effectively used to solve real-world problems in the insurance sector. It provides a smart, efficient, and scalable solution for detecting fraudulent insurance claims and contributes to improving the overall performance of the insurance industry.

CHAPTER 15

REFERENCES

REFERENCES

1. A. A. Khalil et al., “ML Based Method for Insurance Fraud Detection on Class Imbalance Datasets,” IEEE conference, 2024.
2. R. S. P. Rashmi and Y. P. Ganji, “Fraud Detection and Analysis for Insurance Claim Using Machine Learning ,” *International Journal of Scientific Research in Engineering and Management (IJSREM)*, July 2024.
3. V. Sai Sri Nithin, P. Sravani, P. Akash, D. Rishikesh, and Dr. M. V. S. Prasad, “Fraud Detection and Analysis for Insurance Claim Using Machine Learning,” *IJARST*, Vol. 15, Issue 04, April 2025.
4. V. Sai Sri Nithin, P. Sravani, P. Akash, D. Rishikesh, and Dr. M. V. S. Prasad, “Fraud Detection and Analysis for Insurance Claim Using Machine Learning,” *IJARST*, Vol. 15, Issue 04, April 2025.
5. K. Ulaga Priya and S. Pushpa, “A Survey on Fraud Analytics Using Predictive Model
a. in Insurance Claims,” *Int. J. Pure Appl. Math.*, vol. 114, no. 7, pp. 755–767, 2017.
6. E. B. Belhadji, G. Dionne, and F. Tarkhani, “A Model for the Detection of Insurance Fraud,” *Geneva Pap. Risk Insur. Issues Pract.*, vol. 25, no. 4, pp. 517–538, 2000, doi:10.1111/1468-0440.00080.
7. “Predictive Analysis for Fraud Detection.”
8. <https://www.wipro.com/analytics/comparativeanalysis-of-machine-learning-techniques-for->
9. [%0Adetectin/](#).
10. F. C. Li, P. K. Wang, and G. E. Wang, “Comparison of the primitive classifiers with extreme learning machine in credit scoring,” *IEEM 2009 - IEEE Int. Conf. Ind. Eng. Eng. Manag.*, vol. 2, no. 4, pp. 685–688, 2009, doi: 10.1109/IEEM.2009.5373241.
11. V. Khadse, P. N. Mahalle, and S. V. Biraris, “An Empirical Comparison of Supervised Machine Learning Algorithms for Internet of Things Data,” *Proc. - 2018 4th Int. Conf. Comput. Commun. Control Autom. ICCUBEA 2018*, pp. 1–6, 2018, doi: